

**SISTEM INFORMASI MANAJEMEN PENDAFTARAN DAN
PELAPORAN MEDIS PASIEN BERBASIS DESKTOP**

Mata Kuliah: Pemrograman Berorientasi Obyek (24STIE5Y012)

Dosen Pengampu: I Made Sunia Raharja, S.Kom., M.Cs.



Disusun Oleh:

Bunga Makayla Nasywanaya	2505551002
Ni Putu Ayu Sundari	2505551015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS UDAYANA
2026**

KATA PENGATAR

Om Swastiastu,

Puji dan syukur penulis panjatkan ke hadirat Ida Sang Hyang Widhi Wasa, Tuhan Yang Maha Esa, atas limpahan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan tugas besar UTS mata kuliah Pemrograman Berorientasi Objek dengan baik dan tepat waktu, berjudul “Sistem Informasi Manajemen Pendaftaran dan Pelaporan Medis Pasien Berbasis Desktop.”

Penyusunan laporan ini bertujuan untuk memenuhi salah satu syarat tugas akhir serta sebagai sarana pemahaman lebih mendalam mengenai implementasi konsep *Object-Oriented Programming* (OOP) melalui pembuatan sistem informasi yang memiliki fitur CRUD (*Create, Read, Update, Delete*) terintegrasi *database*.

Pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih kepada:

1. Bapak I Made Sunia Raharja, S.Kom., M.Cs. selaku dosen pengampu mata kuliah Pemrograman Berorientasi Objek.
2. Penulis sendiri, yang telah bersungguh-sungguh dan berupaya semaksimal mungkin dalam menyusun dan menyelesaikan tugas akhir ini sesuai ketentuan yang berlaku.

Penulis menyadari bahwa laporan perancangan ini masih jauh dari kata sempurna, baik dari segi isi maupun penyajian. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan demi perbaikan dan penyempurnaan laporan di masa yang akan datang. Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca sebagai tambahan referensi dalam memahami penerapan bahasa pemrograman Java khususnya pada pustaka *Swing* dan konektivitas *database* JDBC.

Om Santih, Santih, Santih Om.

Denpasar, 4 Mei 2026

Penulis

ABSTRAK

Pelayanan kesehatan seperti klinik seringkali menghadapi kendala akibat sistem pendaftaran dan pencatatan data medis yang masih dilakukan secara manual, sehingga proses menjadi lambat, data kurang terorganisir, dan berisiko mengalami kerusakan atau kehilangan. Tugas ini bertujuan untuk mengatasi permasalahan tersebut melalui pengembangan program Sistem Informasi Manajemen Pendaftaran dan Pelaporan Medis Pasien berbasis desktop. Program ini dikembangkan menggunakan bahasa pemrograman Java dengan antarmuka Java *Swing* sebagai bagian dari pemenuhan tugas mata kuliah Pemrograman Berorientasi Objek. Sistem ini dirancang dengan mengimplementasikan fitur CRUD (*Create, Read, Update, Delete*) yang memungkinkan pengelolaan data pasien, pendaftaran, jadwal dokter, serta riwayat medis secara terstruktur. Proses perancangan dilakukan melalui analisa kebutuhan pengguna dan sistem serta menggunakan pemodelan berbagai diagram yang digunakan dalam pengembangan program aplikasi. Hasil dari pengembangan ini berupa aplikasi desktop yang dapat mempermudah proses pendaftaran pasien, meningkatkan efisiensi pengelolaan data medis, serta mendukung pelayanan kesehatan yang lebih terorganisir.

Kata Kunci: Sistem Informasi, Pendaftaran Pasien, Java *Swing*, CRUD, *Database*, Pelayanan Kesehatan, Pemrograman Berorientasi Objek.

DAFTAR ISI

KATA PENGATAR.....	ii
ABSTRAK.....	iii
DAFTAR ISI.....	iv
DAFTAR GAMBAR	vi
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah	1
BAB II PERANCANGAN PROGRAM APLIKASI.....	3
2.1 Analisa Kebutuhan Pengguna.....	3
2.2 Analisa Kebutuhan Sistem	4
2.3 Use Case Diagram	4
2.4 Activity Diagram	6
2.4.1 Activity Diagram Registrasi	6
2.4.2 Activity Diagram Login	7
2.4.3 Activity Diagram My Doctor	8
2.4.4 Activity Diagram Book Appointment	8
2.4.5 Activity Diagram My Appointment	10
2.4.6 Activity Diagram Medical Reports	11
2.4.7 Activity Diagram Doctor Schedules	12
2.4.8 Activity Diagram Registration Approval	13
2.4.9 Activity Diagram Medical Record Entry	14
2.4.10 Activity Diagram Transaction Reports	15
2.5 Entity Relational Diagram	16
2.6 Conceptual Data Model	17
2.7 Physical Data Model	17
2.8 Class Diagram	18

BAB III IMPLEMENTASI DAN PENGUJIAN CRUD	20
3.1 Implementasi Operasi Data (CRUD)	20
3.1.1 Implementasi Proses Create	20
3.1.2 Implementasi Proses Read	24
3.1.3 Implementasi Proses Update	29
3.1.4 Implementasi Proses Delete	34
DAFTAR PUSTAKA	38

DAFTAR GAMBAR

Gambar 2.1	Use Case Diagram Sistem Informasi Pendaftaran Pasien	5
Gambar 2.2	Activity Diagram Registrasi	6
Gambar 2.3	Activity Diagram Login.....	7
Gambar 2.4	Activity Diagram My Doctor	8
Gambar 2.5	Activity Diagram Book Appointment.....	9
Gambar 2.6	Activity Diagram My Appointment.....	10
Gambar 2.7	Activity Diagram Medical Reports.....	11
Gambar 2.8	Activity Diagram Doctor Schedules	12
Gambar 2.9	Activity Diagram Registration Approval.....	13
Gambar 2.10	Activity Diagram Medical Record Entry.....	14
Gambar 2.11	Activity Diagram Transaction Reports	15
Gambar 2.12	ERD Sistem Informasi Pendaftaran Pasien	16
Gambar 2.13	CDM Sistem Informasi Pendaftaran Pasien	17
Gambar 2.14	PDM Sistem Informasi Pendaftaran Pasien.....	18
Gambar 2.15	Class Diagram Sistem Informasi Pendaftaran Pasien.....	19

BAB I

PENDAHULUAN

Bab I Pendahuluan membahas pentingnya peralihan sistem pendaftaran dan pengelolaan data medis dari metode konvensional ke digital untuk meningkatkan efisiensi pelayanan kesehatan. Selain itu, dijelaskan permasalahan seperti antrean fisik serta risiko kerusakan atau kehilangan data akibat sistem manual. Sebagai solusi, diperkenalkan aplikasi sistem informasi berbasis desktop yang membantu proses pendaftaran dan pelaporan medis secara lebih terstruktur dan terintegrasi.

1.1 Latar Belakang Masalah

Kualitas pelayanan kesehatan merupakan faktor penting dalam menunjang kesejahteraan masyarakat sekaligus menjaga efektivitas kerja institusi medis. Namun, pada praktiknya, banyak fasilitas kesehatan masih menghadapi kendala dalam memberikan layanan yang cepat dan efisien karena sistem administrasi yang digunakan belum sepenuhnya terkomputerisasi. Hal ini sejalan dengan temuan dalam *Jurnal PROSISKO* (2025) yang menyebutkan bahwa pengelolaan data yang kurang teratur serta sistem pendaftaran yang belum optimal dapat menyebabkan keterlambatan layanan, penumpukan pasien, hingga menurunnya tingkat kepuasan masyarakat.

Permasalahan tersebut terutama terlihat pada proses pendaftaran yang masih dilakukan secara konvensional. Pasien diharuskan datang langsung dan mengisi formulir secara manual, kemudian data tersebut dicatat kembali oleh petugas ke dalam buku besar. Menurut *Jurnal Ilmu-ilmu Informatika dan Manajemen STMIK* (2018), metode ini tidak hanya menimbulkan antrean panjang yang melelahkan, tetapi juga membuat waktu tunggu menjadi tidak pasti karena pasien tidak dapat memantau urutan antrean secara jelas. Selain itu, penggunaan dokumen berbasis kertas memiliki berbagai risiko, seperti mudah rusak, hilang, serta menyulitkan proses pencarian data ketika dibutuhkan untuk penyusunan laporan medis.

Untuk mengatasi permasalahan tersebut, diperlukan penerapan teknologi informasi yang mampu mengintegrasikan seluruh proses layanan kesehatan dalam

satu sistem yang terstruktur. Penelitian dalam *Jurnal Manajemen Informatika Komputer* (2024) menunjukkan bahwa penggunaan sistem digital dapat mengurangi kesalahan pencatatan, mencegah duplikasi data, serta mempercepat akses informasi secara *real-time* bagi tenaga medis maupun pasien. Hal ini menjadi dasar penting dalam upaya modernisasi sistem pendaftaran agar alur pelayanan menjadi lebih efektif dan efisien.

Berdasarkan hal tersebut, dalam tugas ini dikembangkan sebuah program Sistem Informasi Manajemen Pendaftaran dan Pelaporan Medis Pasien berbasis desktop. Sistem ini dirancang untuk memudahkan pasien dalam melakukan reservasi secara mandiri, mengetahui jadwal dokter dengan lebih jelas, serta mengakses riwayat medis secara digital. Dengan adanya sistem ini, diharapkan fasilitas kesehatan dapat memberikan pelayanan yang lebih tertib, cepat, dan transparan, sehingga kualitas layanan kepada pasien dapat meningkat secara menyeluruh.

BAB II

PERANCANGAN PROGRAM APLIKASI

Bab II Perancangan Program Aplikasi membahas mengenai rencana pembuatan program aplikasi sistem informasi aplikasi berbasis desktop menggunakan Java *Swing* untuk membantu manajemen pendaftaran dan pelaporan medis pasien. Berikut ini adalah penjelasan terkait analisa kebutuhan pengguna dan sistem serta berbagai diagram yang digunakan dalam pengembangan program aplikasi.

2.1 Analisa Kebutuhan Pengguna

Dalam sistem informasi pendaftaran pasien yang diusulkan, terdapat dua pengguna utama yaitu Admin dan Pasien yang dapat berinteraksi serta berkomunikasi di dalam sistem. Kedua pengguna ini memiliki kebutuhan informasi dan fungsi yang berbeda sebagai berikut:

1. Skenario Kebutuhan Admin

Admin memiliki peran sebagai pengelola operasional dan pusat validasi data pendaftaran di klinik. Kebutuhan tersebut meliputi:

- a. Melakukan *login* untuk masuk ke dalam sistem sesuai dengan hak akses yang diberikan.
- b. Mengatur dan memperbarui daftar poliklinik serta jadwal praktik dokter agar informasi yang tersedia bagi pasien selalu aktual.
- c. Meninjau dan mengonfirmasi setiap permohonan janji temu yang dikirimkan oleh pasien melalui aplikasi.
- d. Memasukkan data diagnosa dan resep obat berdasarkan catatan resmi dari dokter setelah pemeriksaan fisik selesai dilakukan.
- e. Menyusun dan mencetak laporan pendaftaran pasien secara berkala untuk keperluan administrasi manajemen klinik.

2. Skenario Kebutuhan Pasien

Pasien merupakan pengguna layanan yang membutuhkan kemudahan dalam melakukan reservasi secara mandiri. Kebutuhan tersebut meliputi:

- a. Melakukan pendaftaran akun baru dengan mengisi identitas diri dan *login* untuk mengakses fitur aplikasi.
- b. Melihat daftar dokter yang tersedia berdasarkan spesialisasi poliklinik beserta waktu praktiknya.
- c. Melakukan pendaftaran secara digital dengan memilih dokter dan menentukan tanggal kunjungan yang diinginkan.
- d. Melihat riwayat janji temu yang telah dibuat serta memantau status persetujuan dari pihak klinik.
- e. Melihat hasil pemeriksaan medis terdahulu, termasuk diagnosa dan resep obat yang telah diproses oleh sistem.

2.2 Analisa Kebutuhan Sistem

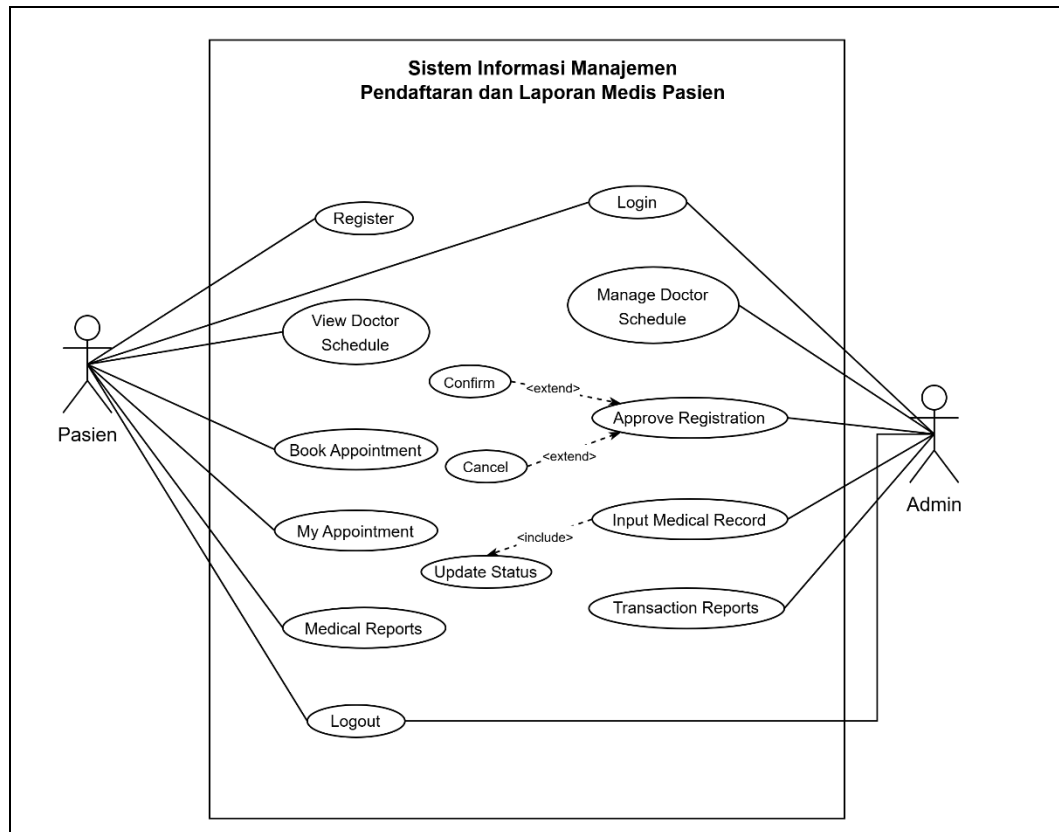
Berdasarkan kebutuhan pengguna di atas, maka sistem informasi yang dibangun harus mampu memenuhi spesifikasi fungsional berikut:

- 1. Dapat menampilkan halaman akses masuk untuk admin serta halaman pendaftaran akun bagi pasien baru.
- 2. Mampu mengolah informasi mengenai data profil pasien, jadwal praktik dokter, hingga catatan kesehatan secara terstruktur.
- 3. Dapat menyimpan permohonan janji temu dari pasien dan memungkinkan pihak admin untuk memperbarui status pendaftaran tersebut secara langsung.
- 4. Dapat menghubungkan data kunjungan pasien dengan hasil diagnosa sehingga informasi rekam medis dapat diakses kembali oleh pengguna yang bersangkutan.
- 5. Dapat menarik seluruh data transaksi pendaftaran pasien rawat jalan untuk kemudian disajikan dalam bentuk laporan fisik yang dapat dicetak.

2.3 Use Case Diagram

Use Case Diagram merupakan gambaran visual yang menunjukkan interaksi antara aktor, yaitu pasien dan admin, dengan fitur-fitur utama yang tersedia dalam sistem informasi pendaftaran. Diagram ini digunakan untuk menjelaskan batasan sistem serta peran masing-masing pengguna dalam

menjalankan fungsi sistem, mulai dari proses masuk ke dalam sistem hingga pengelolaan data pendaftaran dan rekam medis secara terintegrasi.



Gambar 2.1 Use Case Diagram Sistem Informasi Pendaftaran Pasien

Gambar di atas merupakan visualisasi *Use Case Diagram* yang menunjukkan interaksi antara pasien dan admin dengan sistem. Pasien dapat melakukan registrasi, login, melihat jadwal dokter melalui menu *My Doctor*, melakukan pendaftaran pada menu *Book Appointment*, serta melihat riwayat dan hasil pemeriksaan pada menu *My Appointment* dan *Medical Reports*.

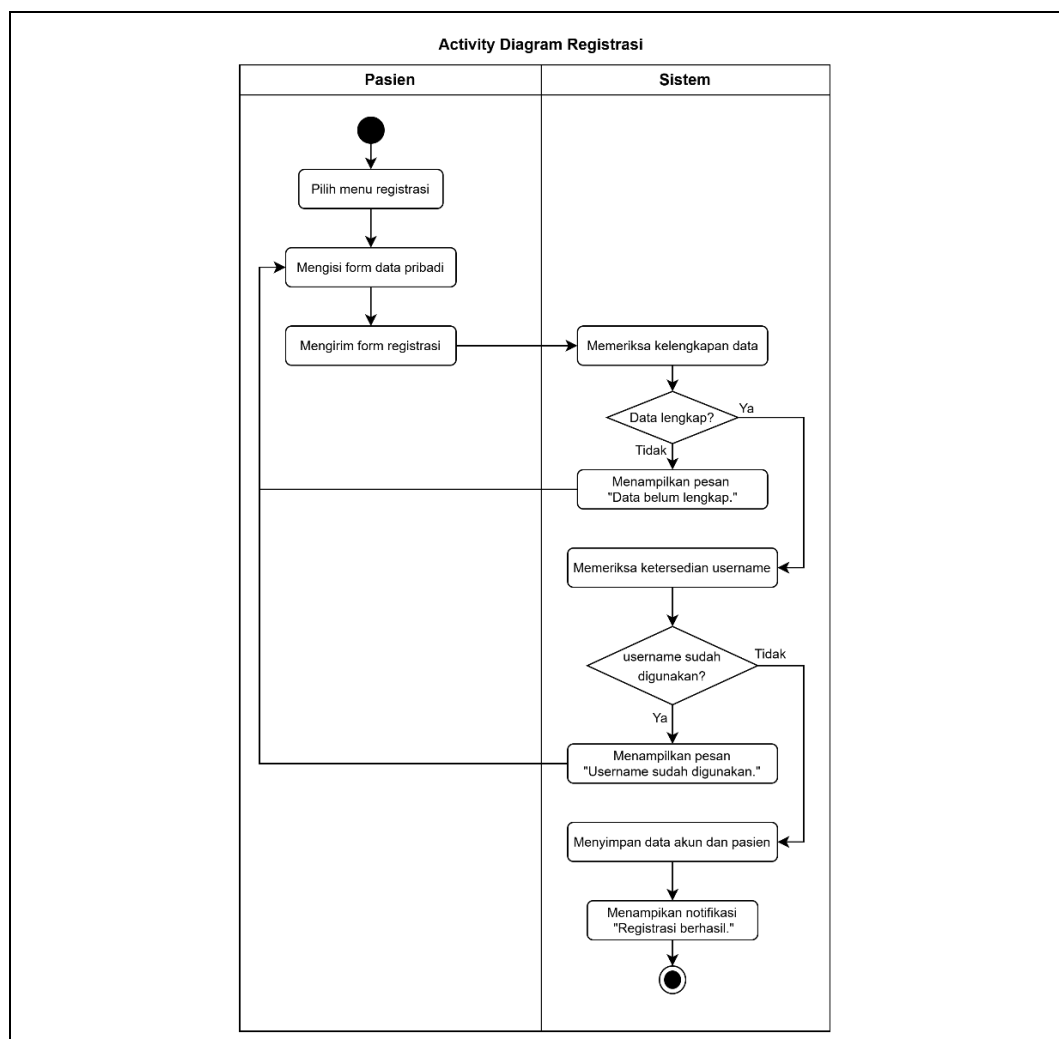
Sementara itu, admin mengelola jadwal dokter melalui *Doctor Schedules*, memproses pendaftaran pasien pada *Registration Approval* dengan status menunggu, dikonfirmasi, atau dibatalkan, serta menginput diagnosa dan resep obat pada *Medical Record Entry* yang sekaligus mengubah status menjadi selesai. Seluruh data kemudian direkap dalam *Transaction Reports* untuk kebutuhan administrasi, sehingga menunjukkan keterkaitan antara aktivitas pasien dan pengelolaan oleh admin dalam sistem.

2.4 Activity Diagram

Activity Diagram merupakan diagram dalam UML yang digunakan untuk menggambarkan alur aktivitas dalam suatu sistem secara terstruktur. Diagram ini menunjukkan urutan proses dari awal hingga akhir, termasuk interaksi antara pengguna dan sistem. Dengan adanya *activity diagram*, alur kerja sistem menjadi lebih mudah dipahami karena divisualisasikan dalam bentuk langkah-langkah yang jelas.

2.4.1 Activity Diagram Registrasi

Activity Diagram Registrasi menggambarkan proses pembuatan akun oleh pasien, mulai dari pengisian data hingga penyimpanan data oleh sistem. Diagram ini juga menunjukkan adanya proses validasi data yang dilakukan oleh sistem sebelum registrasi berhasil.

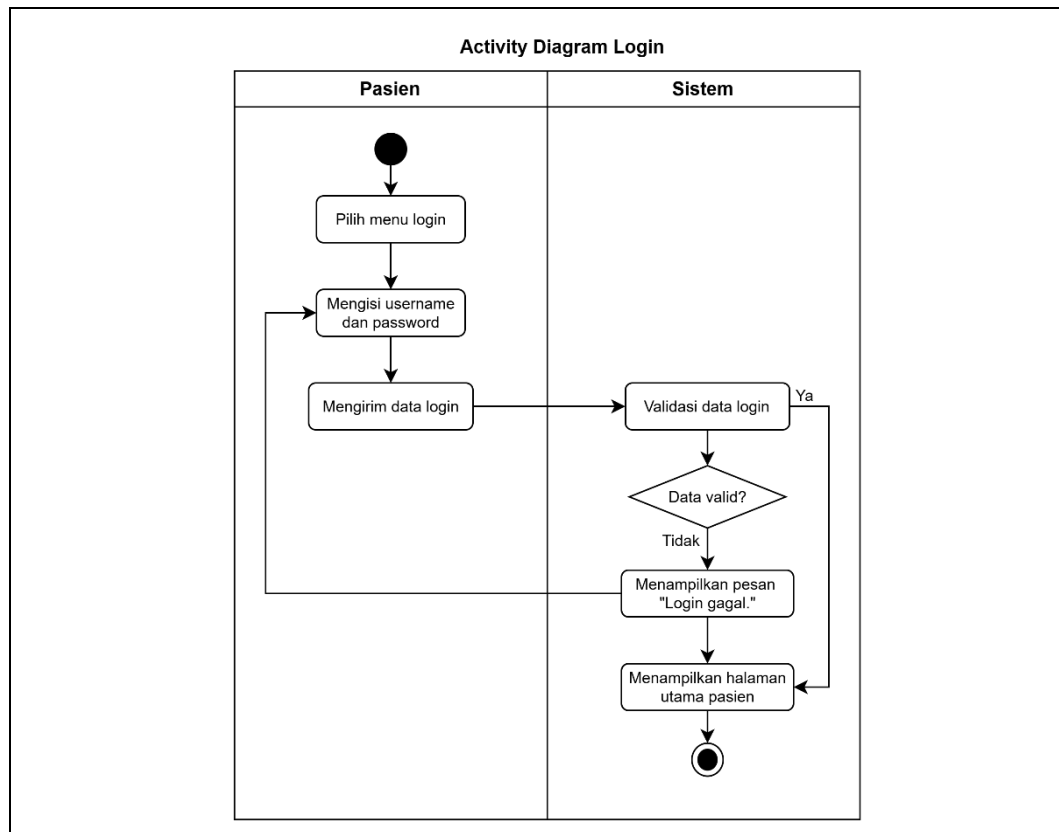


Gambar 2.2 *Activity Diagram* Registrasi

Gambar di atas merupakan alur proses registrasi pada sistem yang dimulai dari pasien mengisi dan mengirim form data pribadi. Sistem kemudian memeriksa kelengkapan data serta ketersediaan username sebelum menyimpan data. Jika terdapat kesalahan, sistem akan menampilkan pesan, sedangkan jika valid, registrasi akan berhasil.

2.4.2 Activity Diagram Login

Activity Diagram Login menggambarkan proses autentikasi pengguna untuk masuk ke dalam sistem berdasarkan *username* dan *password*. Diagram ini juga menunjukkan bagaimana sistem menentukan hak akses pengguna sesuai perannya.



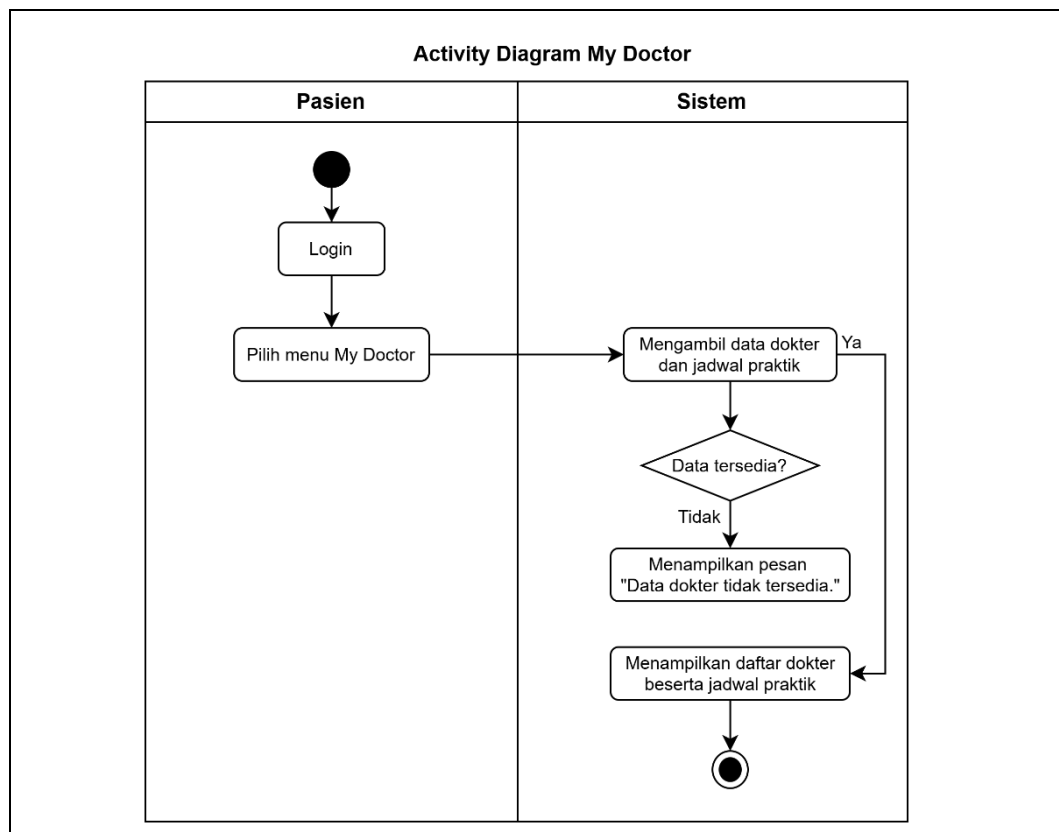
Gambar 2.3 Activity Diagram Login

Gambar di atas merupakan alur *login* yang dimulai dari pengguna memasukkan data login. Sistem akan memvalidasi data tersebut dan menentukan apakah pengguna berhasil masuk atau tidak. Jika valid, pengguna akan diarahkan

ke halaman sesuai perannya, sedangkan jika tidak, sistem menampilkan pesan kesalahan.

2.4.3 Activity Diagram My Doctor

Activity Diagram My Doctor menggambarkan proses pasien dalam melihat informasi dokter dan jadwal praktik yang tersedia. Diagram ini menunjukkan interaksi sederhana antara pasien dan sistem dalam menampilkan data.



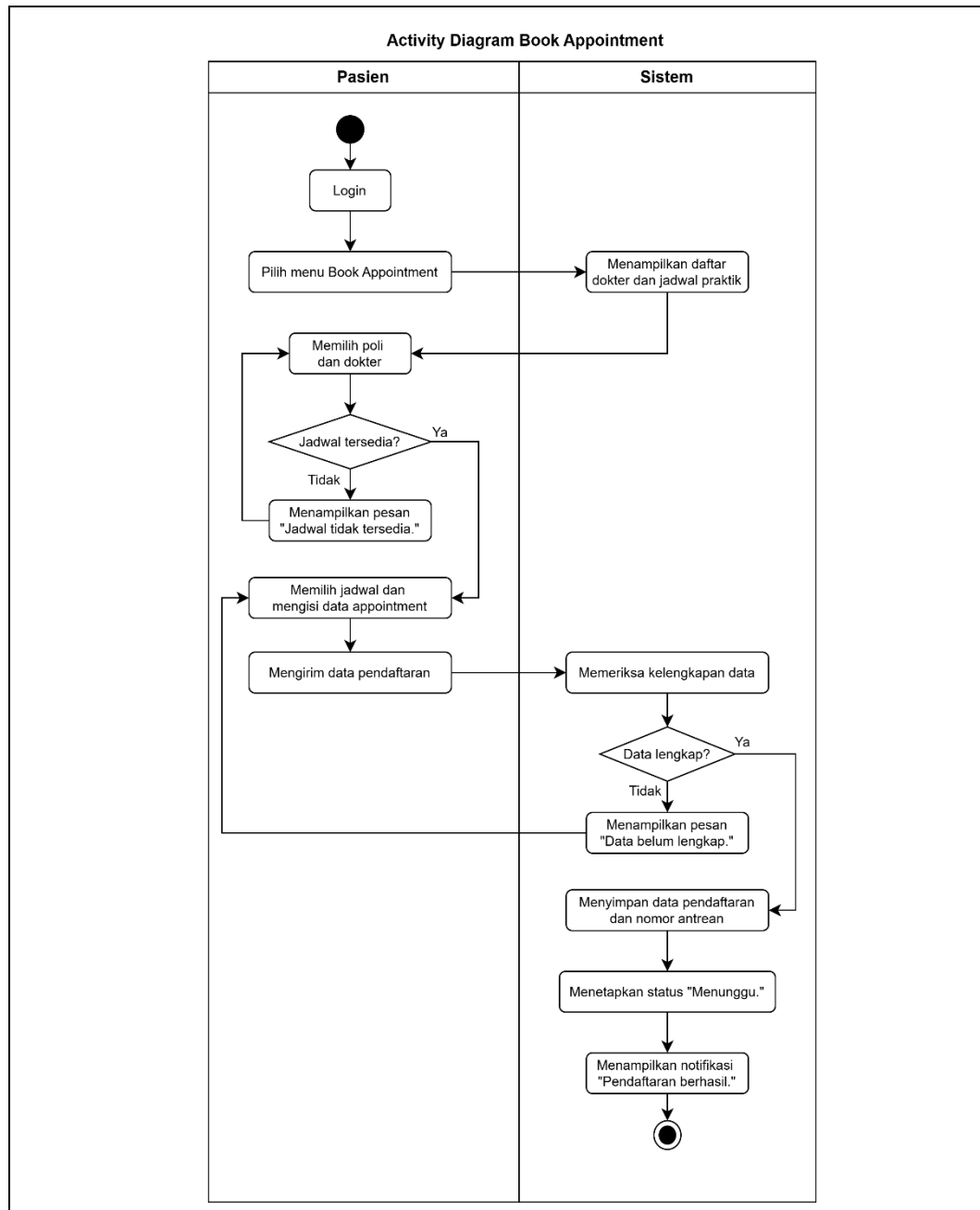
Gambar 2.4 Activity Diagram My Doctor

Gambar di atas merupakan alur ketika pasien memilih menu *My Doctor* untuk melihat daftar dokter. Sistem akan mengambil dan menampilkan data dokter beserta jadwal praktik yang tersedia. Jika data tidak tersedia, sistem akan menampilkan pesan informasi kepada pasien.

2.4.4 Activity Diagram Book Appointment

Activity Diagram Book Appointment menggambarkan proses pasien dalam melakukan pendaftaran kunjungan ke dokter melalui sistem. Proses ini mencakup

pemilihan dokter, pengisian data, hingga sistem memproses dan menyimpan pendaftaran.



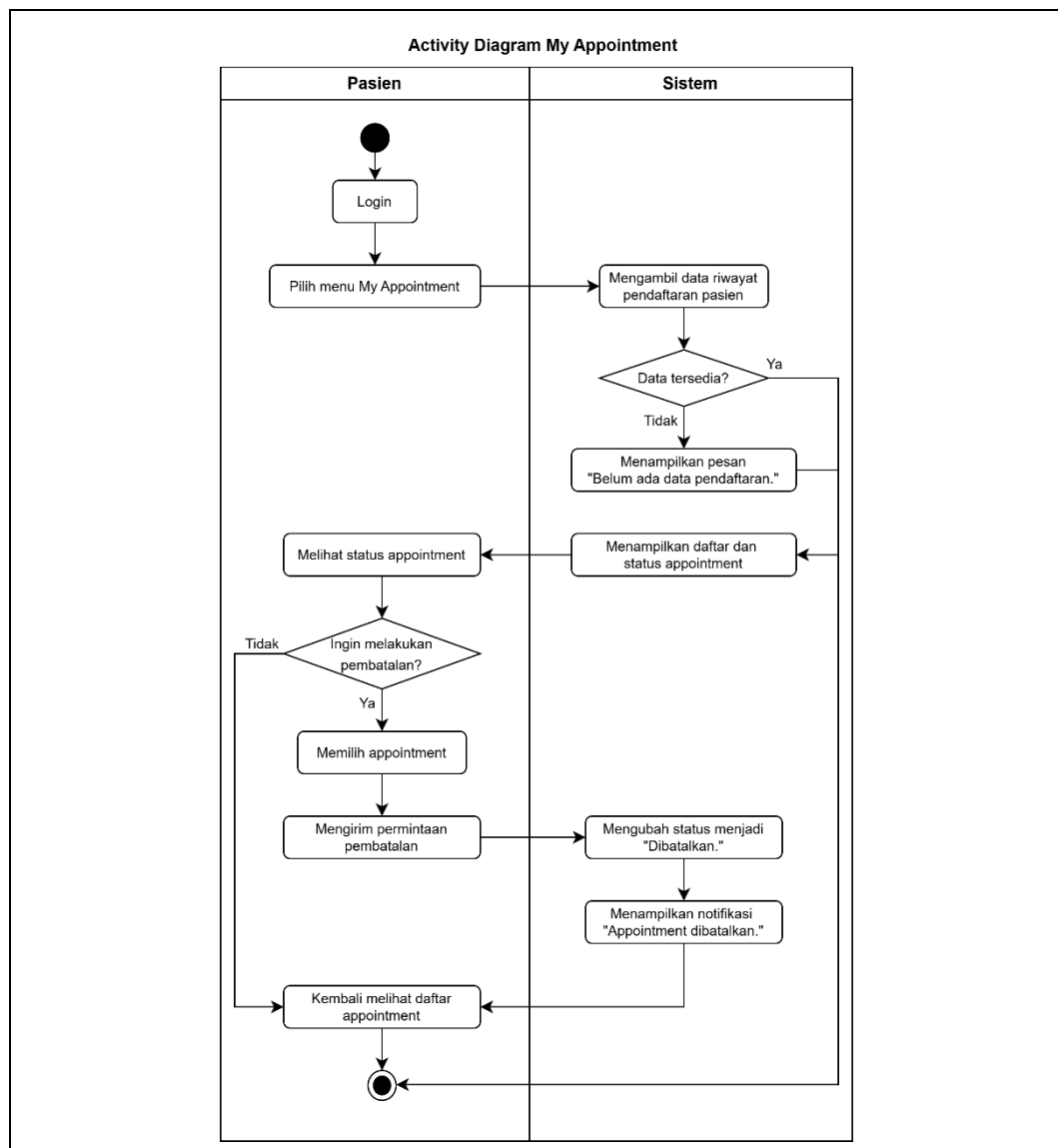
Gambar 2.5 Activity Diagram Book Appointment

Gambar di atas merupakan alur proses pendaftaran kunjungan yang dimulai dari pasien memilih menu *Book Appointment*. Sistem akan menampilkan daftar dokter dan jadwal, kemudian pasien memilih dokter serta mengisi data yang diperlukan. Jika data tidak lengkap atau jadwal tidak tersedia, sistem akan

menampilkan pesan, sedangkan jika valid, sistem akan menyimpan data dan menetapkan status pendaftaran sebagai menunggu.

2.4.5 Activity Diagram My Appointment

Activity Diagram My Appointment menggambarkan proses pasien dalam melihat riwayat pendaftaran serta status kunjungan yang telah dibuat. Diagram ini juga menunjukkan kemungkinan pasien melakukan pembatalan *appointment*.



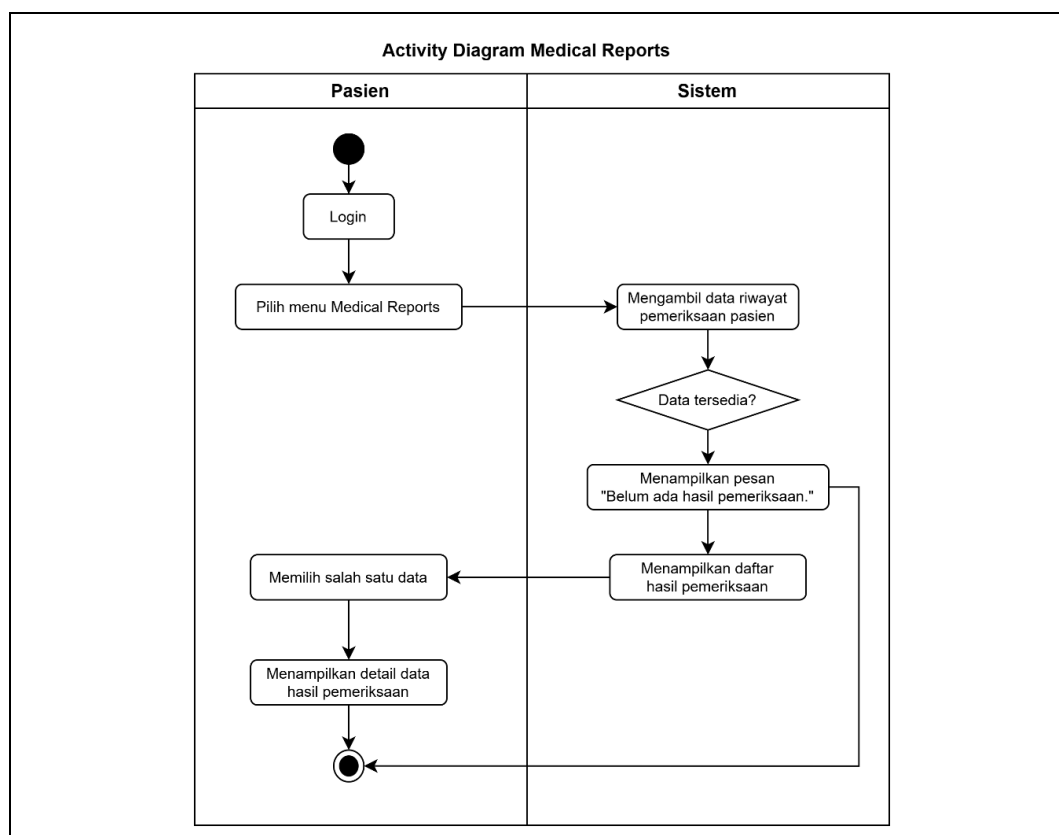
Gambar 2.6 Activity Diagram My Appointment

Gambar di atas merupakan alur ketika pasien mengakses menu *My Appointment* untuk melihat data pendaftaran. Sistem akan menampilkan daftar

appointment beserta statusnya jika tersedia. Pasien juga dapat melakukan pembatalan, sehingga sistem akan memperbarui status menjadi dibatalkan dan menampilkan notifikasi.

2.4.6 Activity Diagram Medical Reports

Activity Diagram Medical Reports menggambarkan proses pasien dalam melihat hasil pemeriksaan medis yang telah dilakukan. Diagram ini mencakup proses pengambilan data hingga penampilan detail diagnosa dan resep obat.

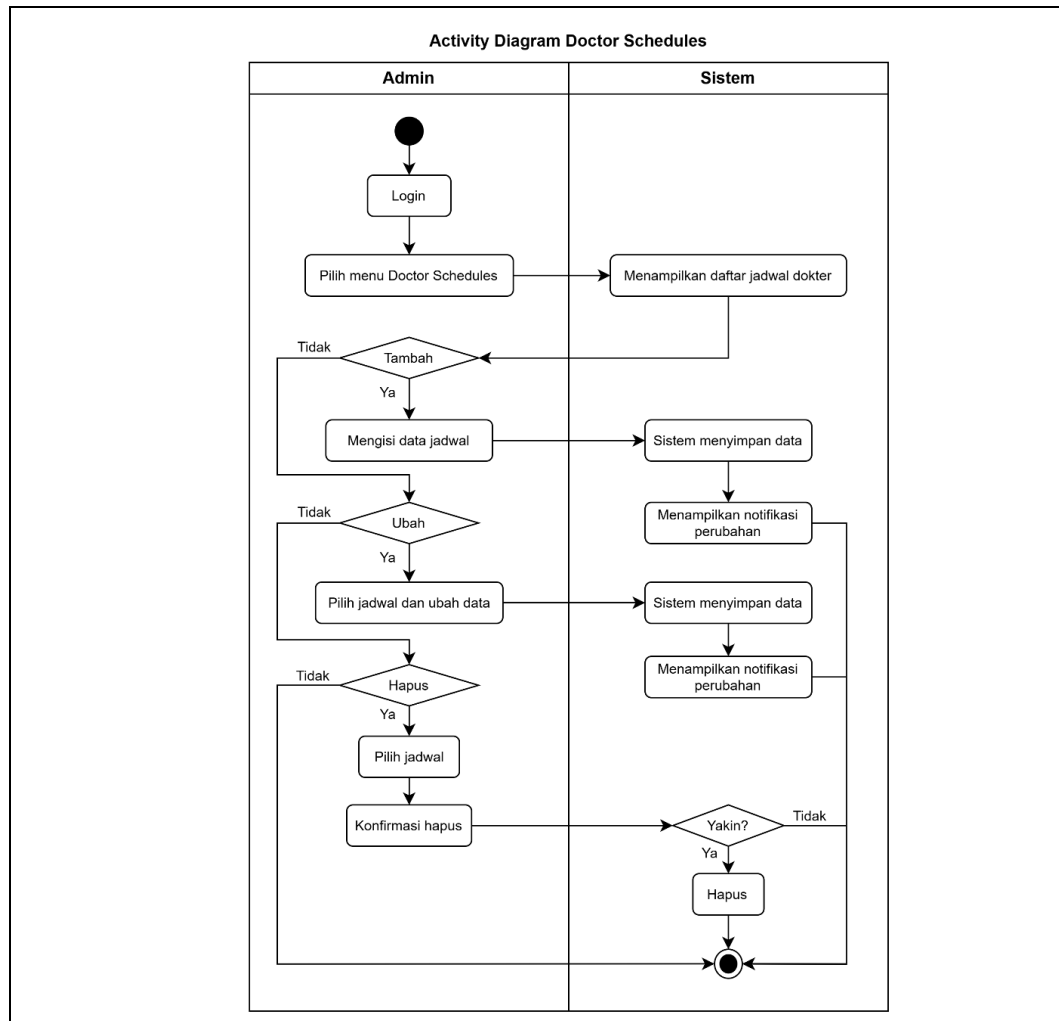


Gambar 2.7 *Activity Diagram Medical Reports*

Gambar di atas merupakan alur ketika pasien memilih menu *Medical Reports* untuk melihat hasil pemeriksaan. Sistem akan menampilkan daftar rekam medis jika tersedia, kemudian pasien dapat memilih salah satu data. Selanjutnya, sistem akan menampilkan detail hasil pemeriksaan berupa diagnosa, obat, dan aturan pakai.

2.4.7 Activity Diagram Doctor Schedules

Activity Diagram Doctor Schedules menggambarkan proses yang dilakukan oleh admin dalam mengelola jadwal praktik dokter. Proses ini mencakup penambahan, perubahan, dan penghapusan data jadwal sesuai kebutuhan.

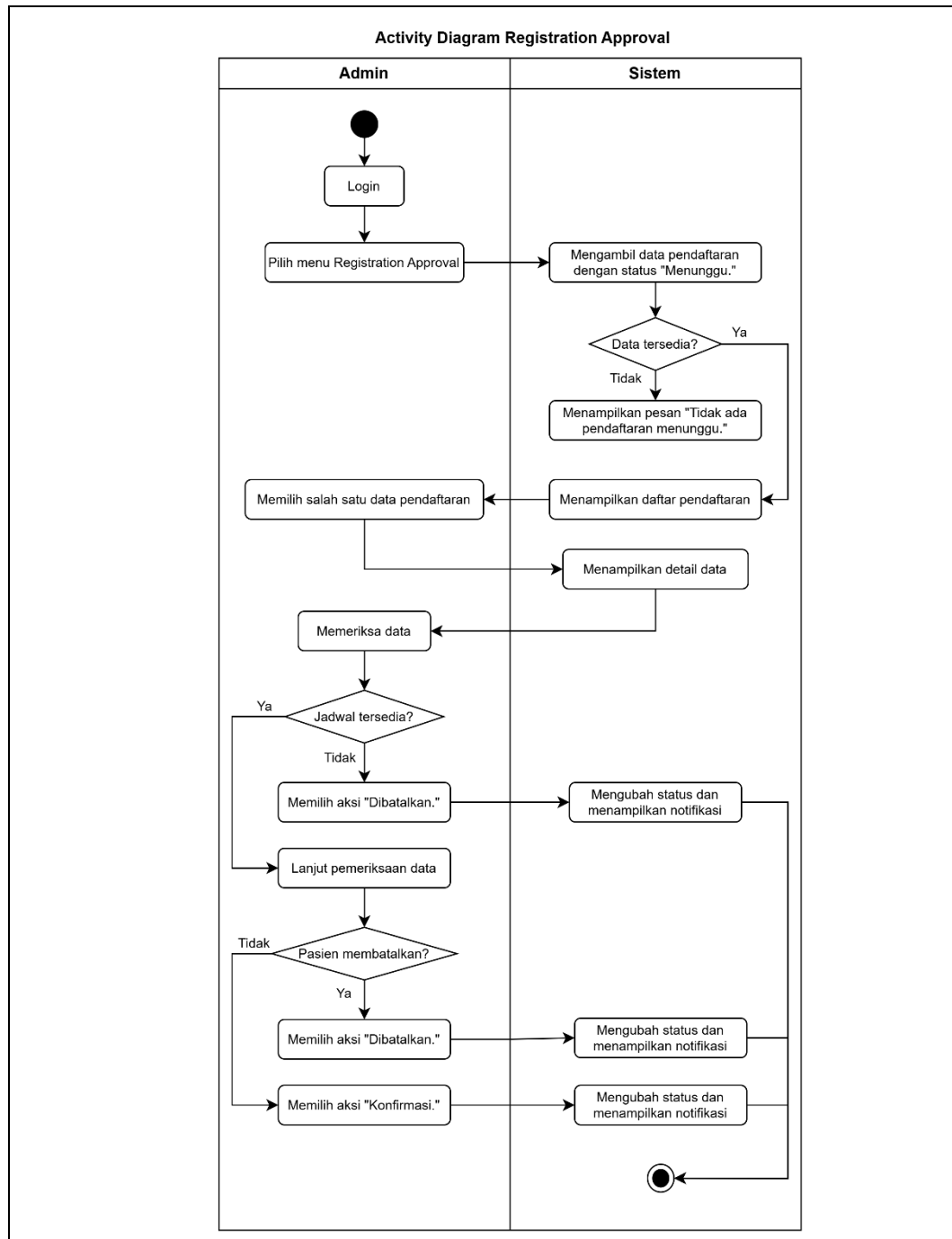


Gambar 2.8 Activity Diagram Doctor Schedules

Gambar di atas merupakan alur pengelolaan jadwal dokter oleh admin melalui menu *Doctor Schedules*. Admin dapat memilih aksi seperti menambah, mengubah, atau menghapus jadwal, kemudian sistem akan memproses sesuai tindakan yang dilakukan. Setelah proses berhasil, sistem akan menampilkan notifikasi sebagai tanda bahwa perubahan data telah dilakukan.

2.4.8 Activity Diagram Registration Approval

Activity Diagram Registration Approval menggambarkan proses admin dalam memverifikasi dan menentukan status pendaftaran pasien. Proses ini mencakup pemeriksaan data serta pengambilan keputusan untuk mengonfirmasi atau membatalkan pendaftaran.

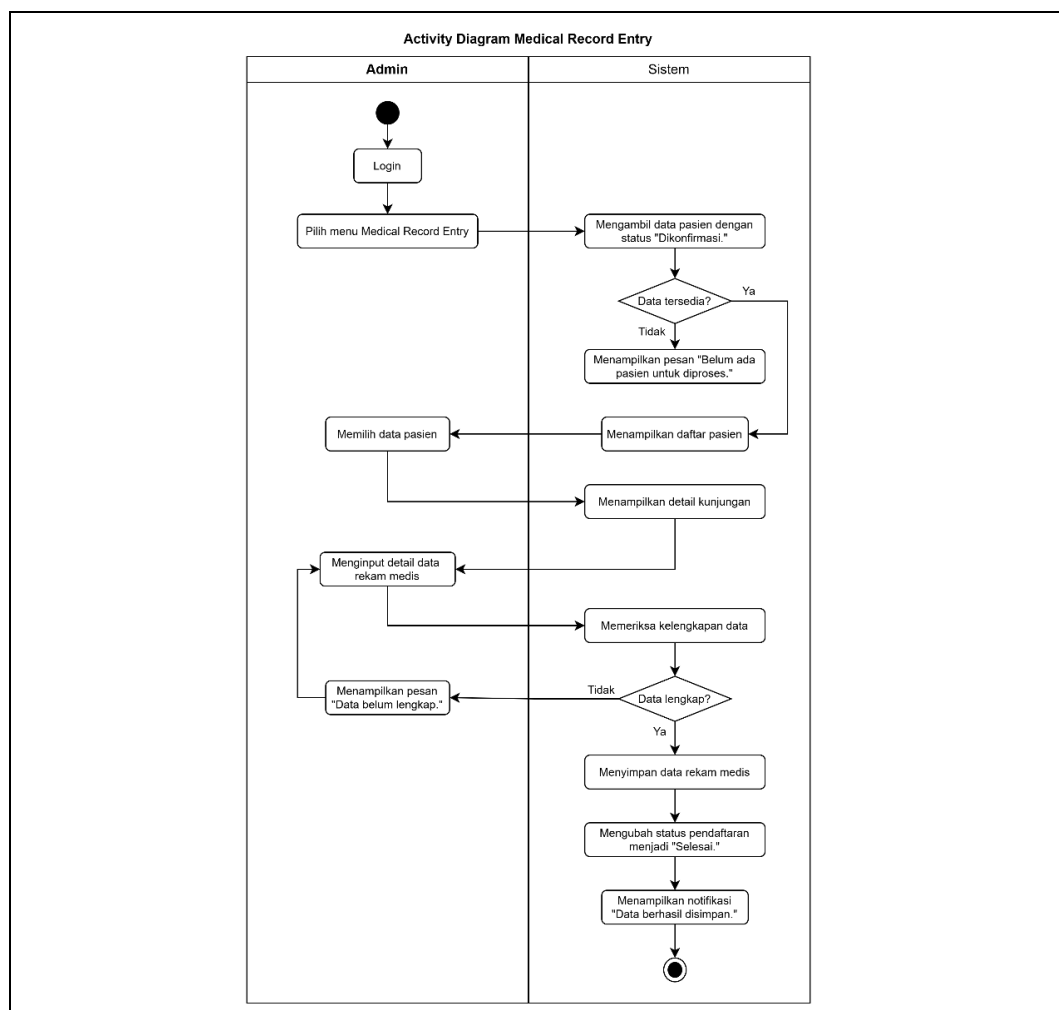


Gambar 2.9 *Activity Diagram Registration Approval*

Gambar di atas merupakan alur proses persetujuan pendaftaran yang dimulai dari admin memilih menu *Registration Approval*. Sistem akan menampilkan data pendaftaran dengan status menunggu, kemudian admin memeriksa detail data. Jika terdapat kendala seperti jadwal tidak tersedia atau pembatalan dari pasien, maka status akan diubah menjadi dibatalkan, sedangkan jika semua sesuai, status akan diubah menjadi dikonfirmasi.

2.4.9 Activity Diagram Medical Record Entry

Activity Diagram Medical Record Entry menggambarkan proses admin dalam menginput hasil pemeriksaan pasien ke dalam sistem. Proses ini dilakukan setelah pendaftaran dikonfirmasi dan mencakup pengisian diagnosa serta resep obat.

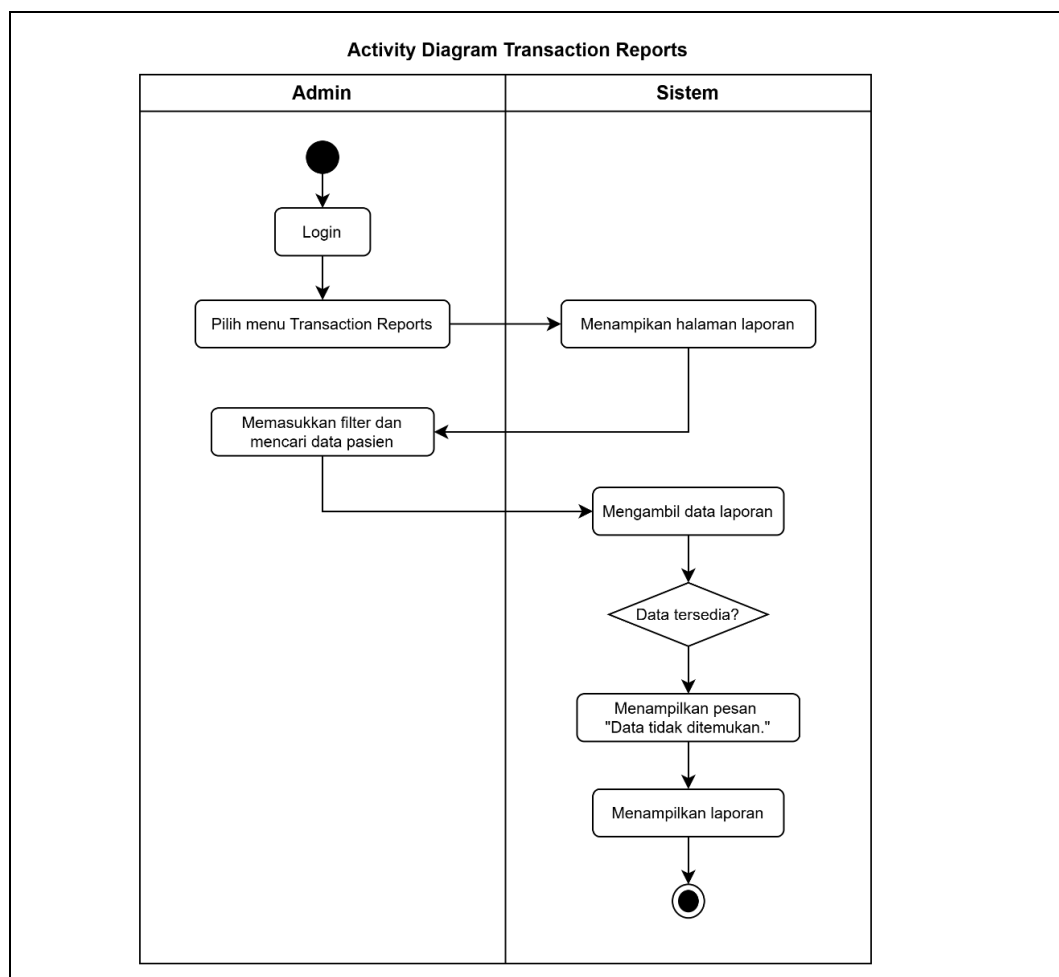


Gambar 2.10 Activity Diagram Medical Record Entry

Gambar di atas merupakan alur proses pengisian rekam medis oleh admin yang dimulai dari memilih pasien dengan status dikonfirmasi. Admin kemudian menginput diagnosa dan data obat, lalu sistem akan memeriksa kelengkapan data. Jika data valid, sistem akan menyimpan informasi dan secara otomatis mengubah status pendaftaran menjadi selesai, sehingga data dapat ditampilkan pada menu *Medical Reports*.

2.4.10 Activity Diagram Transaction Reports

Activity Diagram Transaction Reports menggambarkan proses admin dalam melihat laporan transaksi pendaftaran dan hasil pemeriksaan pasien. Diagram ini berfokus pada penyajian data sebagai kebutuhan informasi administrasi.

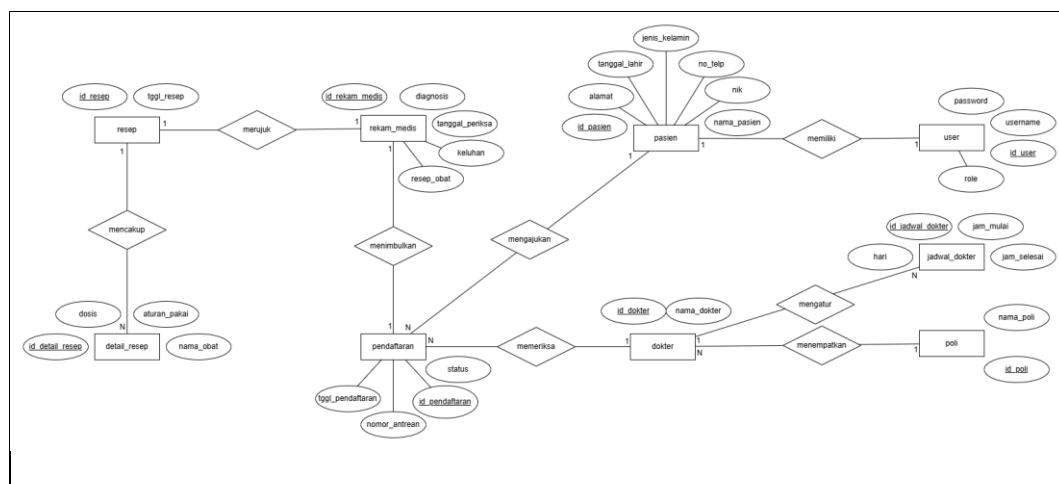


Gambar 2.11 *Activity Diagram Transaction Reports*

Gambar di atas merupakan alur ketika admin mengakses menu *Transaction Reports* untuk melihat laporan. Admin dapat memasukkan filter sesuai kebutuhan, kemudian sistem akan mengambil dan menampilkan data laporan jika tersedia. Jika data tidak ditemukan, sistem akan menampilkan pesan informasi, sedangkan jika tersedia, laporan akan ditampilkan untuk dilihat oleh admin.

2.5 Entity Relational Diagram

Entity Relational Diagram (ERD) adalah model konseptual yang digunakan untuk menggambarkan struktur data dan hubungan antar data di dalam suatu sistem berdasarkan objek-objek dunia nyata. Model ini terdiri dari entitas yang mewakili objek dan relasi yang menjelaskan bagaimana entitas-entitas tersebut saling berhubungan satu sama lain sebelum diimplementasikan ke dalam sistem fisik.

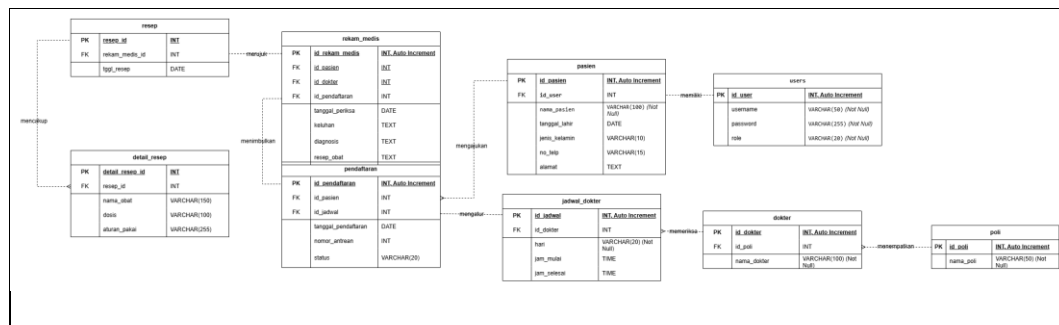


Gambar 2.12 ERD Sistem Informasi Pendaftaran Pasien

Gambar di atas merupakan *Entity Relationship Diagram* (ERD) yang menggambarkan hubungan antar entitas dalam sistem pendaftaran dan rekam medis pasien. Entitas *user* terhubung dengan pasien untuk menyimpan data akun dan identitas, sedangkan dokter terhubung dengan poli serta *jadwal_dokter* untuk menunjukkan informasi layanan. Selain itu, entitas pendaftaran menjadi penghubung utama antara pasien dan dokter, yang kemudian berlanjut ke *rekam_medis*, *resep*, dan *detail_resep* sebagai hasil pemeriksaan.

2.6 Conceptual Data Model

Conceptual Data Model (CDM) adalah konsep yang menggambarkan cara pengguna melihat data yang disimpan dalam basis data. CDM dibuat dalam bentuk tabel tanpa menentukan tipe data secara mendalam pada tahap awal, dan mengilustrasikan hubungan antar tabel untuk digunakan dalam implementasi basis data selanjutnya.

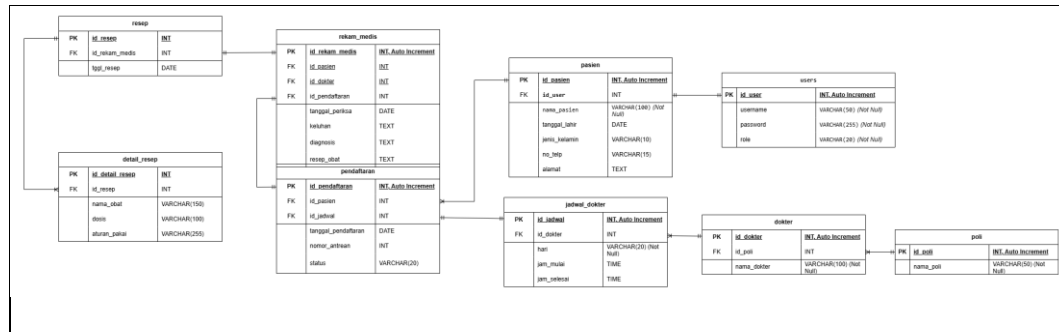


Gambar 2.13 CDM Sistem Informasi Pendaftaran Pasien

Gambar di atas merupakan *Conceptual Data Model* (CDM) yang menggambarkan konsep dasar struktur data tanpa memperhatikan detail teknis implementasi. Pada model ini ditunjukkan entitas beserta atribut utama seperti pasien, dokter, pendaftaran, dan rekam medis serta hubungan antar entitas tersebut. CDM berfungsi sebagai gambaran awal dalam memahami kebutuhan data sebelum dikembangkan ke tahap yang lebih detail.

2.7 Physical Data Model

Physical Data Model (PDM) merupakan model yang merepresentasikan struktur fisik dari basis data yang sudah siap untuk diimplementasikan secara langsung ke dalam perangkat lunak *Database Management System* (DBMS). Berbeda dengan model konseptual, PDM sudah berbentuk tabel-tabel yang dilengkapi keterangan seperti nama tabel, tipe data kolom (seperti INT, VARCHAR, DECIMAL), serta pengaturan kunci-kunci atau *key* pendukung integritas data.

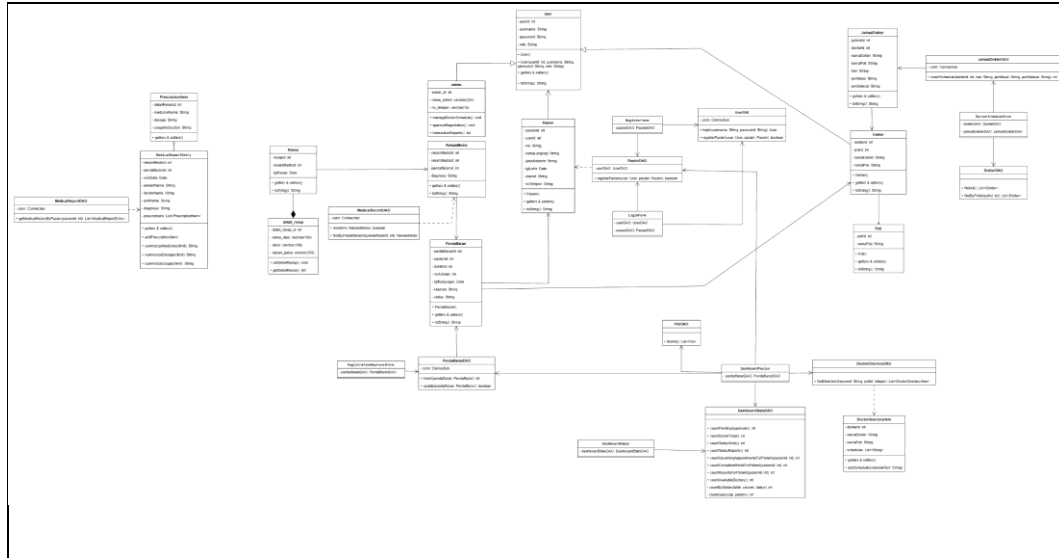


Gambar 2.14 PDM Sistem Informasi Pendaftaran Pasien

Gambar di atas merupakan *Physical Data Model* (PDM) yang menggambarkan implementasi basis data secara detail sesuai dengan sistem yang akan dibangun. Model ini telah mencakup tabel, atribut, tipe data, *primary key*, serta *foreign key* seperti pada *users*, *pasien*, *pendaftaran*, dan tabel lainnya. PDM digunakan sebagai acuan dalam pembuatan database secara langsung agar sesuai dengan kebutuhan sistem yang telah dirancang.

2.8 Class Diagram

Class Diagram merupakan model yang merepresentasikan struktur sistem dengan menggambarkan kelas-kelas, atribut, metode, serta hubungan antar objek di dalam sebuah sistem. Berbeda dengan model data fisik, *Class Diagram* lebih berfokus pada perspektif pemrograman berorientasi objek yang dilengkapi keterangan seperti nama kelas, visibilitas atribut (seperti *private*, *public*), serta metode atau fungsi yang dapat dijalankan. Penggambaran ini juga mencakup pengaturan relasi antar kelas seperti *association*, *aggregation*, atau *composition* guna memastikan integritas logika dan alur data pada tahap perancangan perangkat lunak.



Gambar 2.15 Class Diagram Sistem Informasi Pendaftaran Pasien

Gambar di atas merupakan *Class Diagram* yang menggambarkan struktur logika dan interaksi objek secara detail sesuai dengan sistem yang akan dibangun. Model mencakup berbagai kelas utama, atribut, tipe data, serta *method* atau fungsi operasi seperti pada kelas *user*, pasien, pendaftaran, dan kelas lainnya. *Class Diagram* ini digunakan sebagai acuan dalam implementasi pemrograman berorientasi objek agar relasi antarkomponen sistem, baik itu pewarisan (*inheritance*) maupun keterhubungan antardata, sesuai dengan kebutuhan fungsional yang telah dirancang.

BAB III

IMPLEMENTASI DAN PENGUJIAN CRUD

Bab III Implementasi dan Pengujian CRUD membahas proses penerapan rancangan sistem ke dalam aplikasi desktop yang telah dibuat serta pengujian fungsi-fungsi utamanya. Pembahasan meliputi penjelasan kode program, tampilan antarmuka aplikasi, dan cara kerja sistem dalam mengelola data melalui operasi *Create*, *Read*, *Update*, dan *Delete* (CRUD) pada sistem operasional klinik.

3.1 Implementasi Operasi Data (CRUD)

Aplikasi yang dikembangkan dalam proyek ini merupakan sistem informasi manajemen pendaftaran dan pelaporan medis pasien berbasis desktop. Sistem ini dibangun menggunakan Java dengan antarmuka pengguna berbasis Java Swing serta MySQL sebagai media penyimpanan data. Untuk menghubungkan aplikasi dengan basis data, digunakan arsitektur *Data Access Object* (DAO) yang membantu mengelola proses pengambilan dan penyimpanan data.

Implementasi sistem diuji melalui empat fungsi utama pengelolaan data yang meliputi:

1. Fungsi *Create*, diterapkan pada menu *Book Appointment* untuk membuat janji temu baru dengan dokter.
2. Fungsi *Read*, diterapkan pada menu *Medical Report* untuk menampilkan riwayat rekam medis dan resep obat pasien.
3. Fungsi *Update*, diterapkan pada menu *Registration Approval* yang digunakan admin untuk mengubah status pendaftaran pasien.
4. Fungsi *Delete*, diterapkan pada menu *My Appointment* untuk membatalkan atau menghapus jadwal kunjungan.

Pengujian dilakukan untuk memastikan setiap fitur dapat berjalan dengan baik dan setiap perubahan data yang dilakukan melalui antarmuka aplikasi dapat tersimpan serta ditampilkan dengan benar pada sistem.

3.1.1 Implementasi Proses Create

Implementasi proses *create* merupakan proses penambahan data baru ke dalam sistem sesuai dengan hak akses masing-masing pengguna. Pada sisi admin,

proses *create* terdapat pada menu Doctor Schedule untuk menambahkan jadwal praktik dokter dan *Medical Record Entry* untuk menambahkan data rekam medis pasien. Sementara itu, pada sisi pasien, proses *create* dilakukan saat *Register Account* untuk membuat akun baru serta melalui menu *Book Appointment* untuk melakukan pendaftaran kunjungan. Pada bagian ini, implementasi proses *create* akan dijelaskan melalui menu *Book Appointment*.

3.1.1.1 Potongan Kode Implementasi Book Appointment

Bagian `view` berfungsi sebagai antarmuka yang digunakan pasien untuk melakukan pemesanan jadwal pemeriksaan. Pada menu *Book Appointment*, pasien mengisi data seperti dokter, tanggal kunjungan, dan keluhan. Setelah tombol pemesanan dipilih, sistem memvalidasi data, menentukan nomor antrean secara otomatis, kemudian membentuk sebuah *object* Pendaftaran yang akan dikirim ke *class* PendaftaranDAO untuk disimpan ke *database*.

```
private void bookAppointment() {
// ... [Bagian validasi input komponen UI] ...

try {
// 1. Mengonversi tanggal dan menghitung nomor antrean otomatis
Date sqlDate = Date.valueOf(visitDate);
int queueNumber =
pendaftaranDAO.getNextQueueNumber(doctor.getDokterId(),
sqlDate);

// 2. Proses instansiasi class dan pengisian data object model
(CREATE MEMORI)
Pendaftaran pendaftaran = new Pendaftaran();
pendaftaran.setPasienId(currentPasien.getPasienId());
pendaftaran.setDokterId(doctor.getDokterId());
pendaftaran.setNoAntrean(queueNumber);
pendaftaran.setTglKunjungan(sqlDate);
pendaftaran.setKeluhan(complaint);
pendaftaran.setStatus("Menunggu");

// 3. Mengeksekusi data object ke lapisan database via dao
pendaftaranDAO.insert(pendaftaran);
JOptionPane.showMessageDialog(this, "Appointment booked
successfully.", "Success", JOptionPane.INFORMATION_MESSAGE);
resetForm();
} catch (RuntimeException ex) {
JOptionPane.showMessageDialog(this, ex.getMessage(), "Database
Error", JOptionPane.ERROR_MESSAGE);
}
```

Kode Program 3.1 Implementasi Proses CREATE pada Book Appointment

Kode Program di atas merupakan implementasi proses *create* pada bagian *view*. Data yang diinput pasien diambil dari komponen antarmuka, kemudian

disimpan ke dalam *object* Pendaftaran menggunakan *method* setter. Setelah seluruh data berhasil diisi, sistem memanggil *method* `pendaftaranDAO.insert(pendaftaran)` untuk melanjutkan proses penyimpanan data ke *database*. Selain itu, sistem juga menghasilkan nomor antrian secara otomatis sesuai dokter dan tanggal kunjungan yang dipilih.

Object Pendaftaran yang telah dibentuk sebelumnya menggunakan *class* Pendaftaran sebagai model untuk menyimpan seluruh data pendaftaran selama proses berlangsung. *Class* ini memuat atribut yang sesuai dengan struktur tabel `tb_pendaftaran`, seperti identitas pasien, dokter, nomor antrian, tanggal kunjungan, keluhan, dan status pendaftaran.

```
public class Pendaftaran {
    private int pendaftaranId;
    private int pasienId;
    private int dokterId;
    private int noAntrean;
    private java.sql.Date tglKunjungan;
    private String keluhan;
    private String status;

    // [Method Constructor, Getter, dan Setter dideklarasikan di sini]
}
```

Kode Program 3.2 Implementasi Class Pendaftaran sebagai Model

Kode Program di atas merupakan implementasi *class* Pendaftaran yang digunakan sebagai model dalam proses pendaftaran pasien. Seluruh atribut dideklarasikan menggunakan akses *private* sehingga hanya dapat diakses melalui *method* getter dan setter. Penerapan konsep enkapsulasi membantu menjaga konsistensi data sebelum diteruskan ke proses penyimpanan.

Setelah data berhasil disimpan ke dalam *object* Pendaftaran, proses berikutnya dilakukan oleh *class* PendaftaranDAO yang bertugas menghubungkan aplikasi dengan *database*. *Class* ini menerima *object* dari bagian View, kemudian menjalankan perintah SQL untuk menambahkan data ke tabel `tb_pendaftaran`.

```
public int insert(Pendaftaran pendaftaran) {
    // 1. Menyiapkan kueri sql insert
    String sql = "INSERT INTO tb_pendaftaran (pasien_id,
dokter_id, no_antrean, tgl_kunjungan, keluhan, status) "
        + "VALUES (?, ?, ?, ?, ?, ?)";
```

```

        // 2. Membuka koneksi database dan membuat prepared statement
        try (Connection conn = DatabaseConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql,
                PreparedStatement.RETURN_GENERATED_KEYS)) {

// 3. Proses parameter binding (mengisi tanda tanya dengan data
// dari object model)
            ps.setInt(1, pendaftaran.getPasienId());
            ps.setInt(2, pendaftaran.getDokterId());
            ps.setInt(3, pendaftaran.getNoAntrean());
            ps.setDate(4, pendaftaran.getTglKunjungan());
            ps.setString(5, pendaftaran.getKeluhan());
            ps.setString(6,
                statusForDatabase(pendaftaran.getStatus()));

// 4. Eksekusi perintah sql ke server mysql
            int affectedRows = ps.executeUpdate();
            if (affectedRows == 0) {
                throw new SQLException("Creating appointment failed,
no rows affected.");
            }

// 5. Mengambil id primary key yang di-generasi otomatis oleh
// database
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            }

            throw new SQLException("Creating appointment failed, no
ID obtained.");
        } catch (SQLException e) {
            throw new RuntimeException("Failed to insert appointment:
" + e.getMessage(), e);
        }
    }
}

```

Kode Program 3.3 Implementasi Method Insert pada PendaftaranDAO

Kode Program di atas merupakan implementasi *method* `insert()` pada *class* `PendaftaranDAO` yang digunakan untuk menyimpan data pendaftaran ke *database*. *Method* ini memanfaatkan `PreparedStatement` untuk mengisi parameter pada perintah SQL `INSERT INTO` menggunakan data dari *object* `Pendaftaran`. Setelah *method* `executeUpdate()` dijalankan, data akan tersimpan ke dalam *database* dan sistem memperoleh id pendaftaran yang dibuat secara otomatis (*auto increment*).

3.1.1.2 Tampilan Interface Book Appointment dan Pengujian Create

Menu *Book Appointment* digunakan oleh pasien untuk melakukan pendaftaran pemeriksaan secara mandiri melalui aplikasi. Pada menu ini, pasien

diminta mengisi data yang diperlukan, seperti memilih poliklinik, dokter beserta jadwal praktik yang tersedia, tanggal kunjungan, dan keluhan yang dialami.

CarePoint - Patient Dashboard

Easily secure your appointment queue with our specialists.

Book Appointment

Patient Name
Valerie Clarissa

Poli
Poli Umum

Doctor
dr. Elio Sebastian - Poli Umum

Schedule
Senin | 08:00:00 - 12:00:00

Visit Date
2026-07-06

Complaint
Migrain parah dan leher kaku

Book Appointment Reset

Booking Summary

Poli: Poli Umum
Doctor: dr. Elio Sebastian
Schedule: Senin | 08:00:00 - 12:00:00
Visit Date: 2026-07-06
Next Queue: 1

Appointment will be saved with status Pending.

Gambar 3.1 Tampilan Interface Book Appointment

Gambar di atas merupakan tampilan menu *Book Appointment* yang telah diisi oleh pasien atas nama Valerie Clarissa. Setelah pasien mengisi kolom dan mengirimkan data maka proses pendaftaran berhasil dilakukan, sistem akan menyimpan data pendaftaran, menghasilkan nomor antrean secara otomatis, serta menetapkan status pendaftaran menjadi "Menunggu" sebagai tanda bahwa data telah berhasil ditambahkan dan menunggu proses konfirmasi oleh admin.

3.1.2 Implementasi Proses Read

Implementasi proses *read* digunakan untuk menampilkan data yang telah tersimpan di dalam sistem sesuai dengan kebutuhan pengguna. Pada sisi pasien, proses *read* terdapat pada menu *My Doctor*, *My Appointment*, dan *Medical Reports*. Sedangkan pada sisi admin, proses *read* dilakukan melalui menu *Registration Approval*, *Medical Record Entry*, dan *Transaction Reports*. Pada bagian ini, implementasi proses *read* akan dijelaskan melalui menu *Medical Reports*.

3.1.2.1 Potongan Kode Implementasi Medical Report

Bagian *view* berfungsi sebagai antarmuka yang digunakan pasien untuk melihat riwayat hasil pemeriksaan yang telah dilakukan. Pada menu *Medical*

Reports, sistem mengambil data rekam medis berdasarkan akun pasien yang sedang *login*, kemudian menampilkan informasi seperti tanggal kunjungan, poliklinik, dokter, diagnosa, dan ringkasan obat ke dalam bentuk tabel sehingga riwayat pemeriksaan dapat dilihat dengan mudah.

```
private void refreshData() {
    reports.clear();
    tableModel.setRowCount(0);

    if (currentPasien == null) {
        renderEmptyState();
        return;
    }

    // 1. Mengambil data rekam medis berdasarkan id pasien
    reports.addAll(medicalReportDAO.findByPasienId(currentPasien.getPasienId()));

    if (reports.isEmpty()) {
        renderEmptyState();
        return;
    }

    // 2. Menampilkan data ke dalam tabel
    for (MedicalReportEntry entry : reports) {
        tableModel.addRow(new Object[] {
            entry.getRekamMedisId(),
            formatDate(entry.getVisitDate()),
            safeText(entry.getDoctorName()),
            safeText(entry.getPoliName()),
            safeText(entry.getDiagnosis()),
            entry.summarizeMedicines(Integer.MAX_VALUE),
            entry.summarizeDosages(Integer.MAX_VALUE),
            entry.summarizeUsages(Integer.MAX_VALUE)
        });
    }

    // 3. Menampilkan ringkasan data terbaru
    renderSummary(reports.get(0));
    renderRecentReport(reports.get(0));
}
```

Kode Program 3.4 Implementasi Proses Read pada Medical Report

Kode Program di atas merupakan implementasi proses *read* pada bagian *view*. Sistem mengambil data rekam medis pasien melalui *method* `medicalReportDAO.findByPasienId()`, kemudian menampilkan setiap data ke dalam tabel menggunakan perulangan *for-each*. Selain itu, sistem juga menampilkan ringkasan rekam medis terbaru sehingga informasi pemeriksaan pasien dapat dilihat dengan lebih mudah.

Object yang digunakan untuk menampung data rekam medis menggunakan *class* `MedicalReportEntry` sebagai model. *Class* ini memuat informasi hasil pemeriksaan pasien, seperti tanggal kunjungan, nama dokter, poliklinik, diagnosa, serta daftar obat yang diperoleh dari hasil pemeriksaan.

```
public class MedicalReportEntry {

    private int rekamMedisId;
    private int pendaftaranId;
    private Date visitDate;
    private String patientName;
    private String doctorName;
    private String poliName;
    private String diagnosis;
    private final List<PrescriptionItem> prescriptions = new
    ArrayList<>();

    // [Method Getter, Setter, dan Fungsi Helper Ringkasan Obat
    dideklarasikan di sini]
}
```

Kode Program 3.5 Implementasi Class `MedicalReportEntry` sebagai Model

Kode Program di atas merupakan implementasi *class* `MedicalReportEntry` yang digunakan sebagai model dalam proses menampilkan riwayat rekam medis pasien. Seluruh atribut dideklarasikan menggunakan akses *private* sehingga hanya dapat diakses melalui *method* *getter* dan *setter*. Selain menyimpan informasi pemeriksaan, *class* ini juga memiliki atribut `List<PrescriptionItem>` yang digunakan untuk menampung lebih dari satu data obat pada setiap rekam medis.

Setelah data ditampung pada *object* `MedicalReportEntry`, proses berikutnya dilakukan oleh *class* `MedicalReportDAO` yang bertugas mengambil data dari *database*. *Class* ini menjalankan perintah SQL untuk membaca data rekam medis berdasarkan id pasien, kemudian mengembalikan hasilnya ke bagian *view* agar dapat ditampilkan kepada pengguna.

```
private String baseSql() {
    return "SELECT rm.rekam_medis_id, rm.pendaftaran_id,
    p.tgl_kunjungan, "
        + "pas.nama_lengkap AS pasien_nama, d.nama_lengkap
    AS dokter_nama, pol.nama_poli, "
        + "rm.diagnosa, dr.detail_resep_id, m.nama_obat,
    dr.dosis, dr.aturan_pakai "
        + "FROM tb_rekam_medis rm "
```



```

        + "JOIN tb_pendaftaran p ON rm.pendaftaran_id =
p.pendaftaran_id "
        + "JOIN tb_pasien pas ON p.pasien_id = pas.pasien_id
"
        + "JOIN tb_dokter d ON p.dokter_id = d.dokter_id "
        + "JOIN tb_poliklinik pol ON d.poli_id = pol.poli_id
"
        + "LEFT JOIN tb_resep r ON rm.rekam_medis_id =
r.rekam_medis_id "
        + "LEFT JOIN tb_detail_resep dr ON r.resep_id =
dr.resep_id "
        + "LEFT JOIN tb_obat m ON dr.obat_id = m.obat_id ";
    }

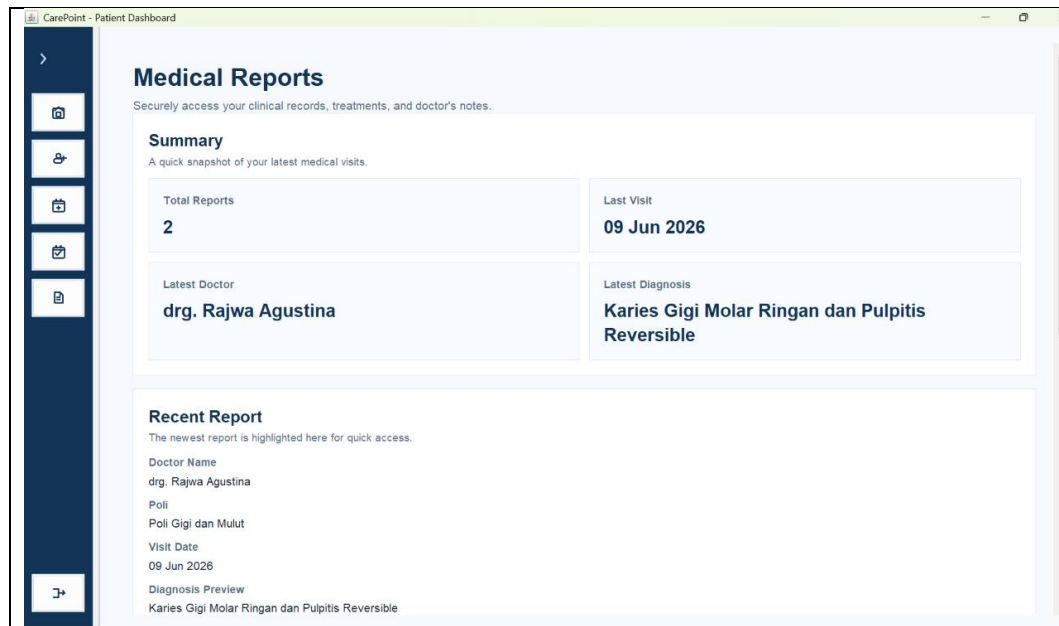
```

Kode Program 3.6 Implementasi Method `findByPasienId` pada `MedicalReportDAO`

Kode Program di atas merupakan implementasi *method* `findByPasienId()` pada *class* `MedicalReportDAO` yang digunakan untuk mengambil data rekam medis berdasarkan id pasien. *Method* ini menjalankan perintah SQL menggunakan `JOIN` dan `LEFT JOIN` untuk menggabungkan data dari beberapa tabel yang saling berelasi, seperti `tb_rekam_medis`, `tb_pendaftaran`, `tb_pasien`, `tb_dokter`, `tb_poli`, `tb_resep`, dan `tb_detail_resep`. Data yang berhasil diperoleh kemudian dikembalikan dalam bentuk `List<MedicalReportEntry>` sehingga dapat ditampilkan pada halaman *Medical Reports*.

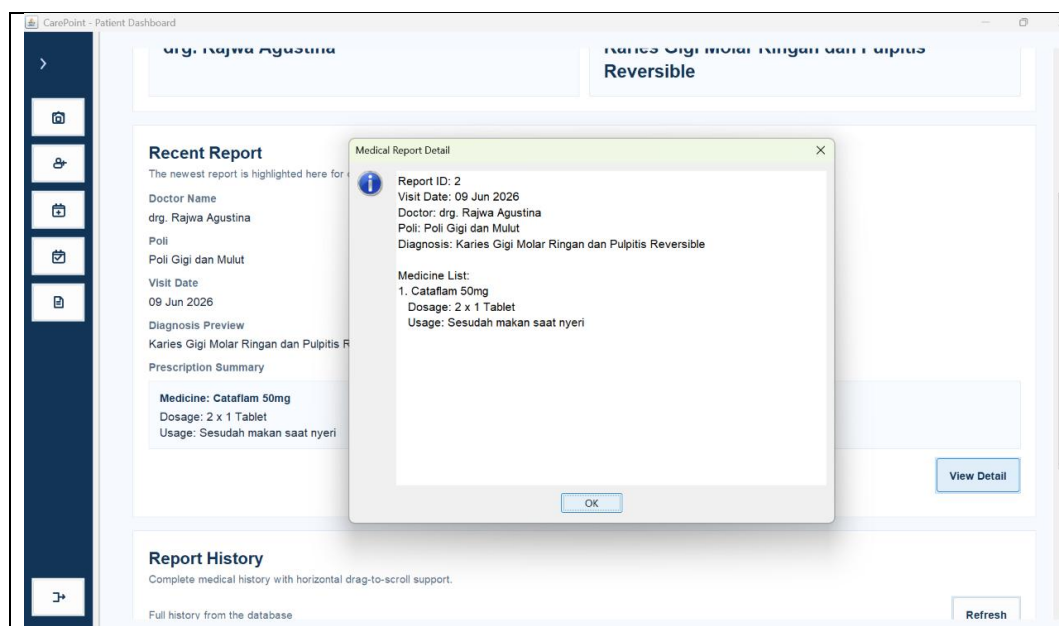
3.1.2.2 Tampilan Interface Medical Report dan Pengujian Read

Menu *Medical Reports* digunakan untuk menampilkan riwayat rekam medis pasien yang telah melakukan pemeriksaan. Pada pengujian ini digunakan akun pasien Adriana Paula yang telah memiliki beberapa riwayat kunjungan, sehingga sistem dapat menampilkan informasi hasil pemeriksaan, diagnosa, dan resep obat sesuai dengan data yang tersimpan pada database.



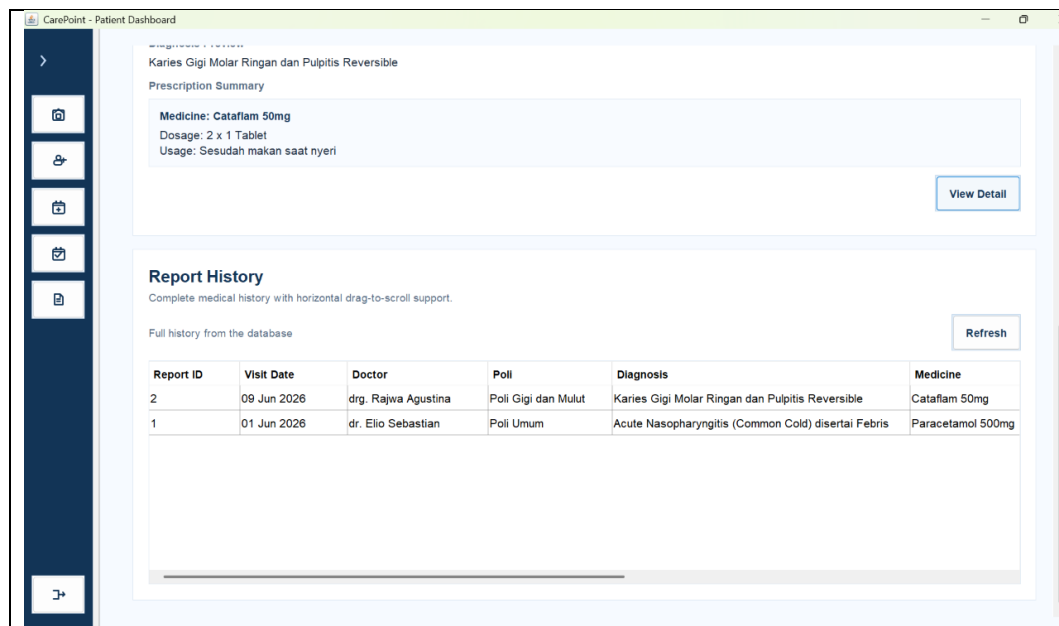
Gambar 3.2 Tampilan Panel Summary pada Medical Reports

Gambar di atas merupakan bagian panel *summary* pada tampilan utama halaman *Medical Reports* yang menampilkan ringkasan informasi rekam medis pasien. Pada bagian ini sistem menampilkan jumlah total kunjungan, tanggal kunjungan terakhir, serta poliklinik terakhir yang dikunjungi berdasarkan riwayat pemeriksaan pasien Adriana Paula. Informasi yang ditampilkan merupakan hasil kunjungan paling terbaru.



Gambar 3.3 Tampilan Pop-up View Detail pada Medical Reports

Gambar di atas merupakan bagian panel *Recent Report* pada halaman utama *Medical Reports* yang dapat menampilkan *reports* terakhir pasien melalui fitur *pop-up View Detail* yang muncul. *Pop-up* ini menampilkan informasi rekam medis secara lengkap, meliputi tanggal kunjungan, nama dokter, poliklinik, diagnosa, serta rincian resep obat beserta dosis dan aturan pakainya.



Gambar 3.4 Tampilan Panel Report History

Gambar di atas merupakan tampilan panel *Report History* yang berisi seluruh hasil pemeriksaan pasien dalam bentuk tabel riwayat rekam medis. Hasil pengujian menunjukkan bahwa proses *read* telah berjalan dengan baik karena seluruh riwayat rekam medis berhasil ditampilkan sesuai data yang tersimpan pada *database*.

3.1.3 Implementasi Proses Update

Implementasi proses *update* digunakan untuk memperbarui data yang telah tersimpan agar tetap sesuai dengan kondisi terbaru. Pada sisi admin, proses *update* terdapat pada menu *Doctor Schedule* untuk mengubah jadwal praktik dokter dan *Registration Approval* untuk memperbarui status pendaftaran pasien. Sementara itu, pada sisi pasien, proses *update* dilakukan melalui menu *My Appointment* ketika pasien membatalkan *appointment* sehingga status pendaftaran

berubah menjadi Dibatalkan. Pada bagian ini, implementasi proses *update* akan dijelaskan melalui menu *Registration Approval*.

3.1.3.1 Potongan Kode Implementasi Perubahan Status Antrean

Bagian *view* berfungsi sebagai antarmuka yang digunakan admin untuk mengelola status pendaftaran pasien pada menu *Registration Approval*. Admin memilih data pendaftaran yang akan diproses, kemudian menentukan status baru, seperti Dikonfirmasi atau Dibatalkan. Setelah status dipilih, sistem akan memperbarui data pada *database* dan menampilkan informasi terbaru pada halaman.

```
private void changeStatus(String status) {
    if (selectedPendaftaranId <= 0) {
        JOptionPane.showMessageDialog(this,
            "Please select an appointment first.",
            "Validation",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    try {
        boolean updated =
pendaftaranDAO.updateStatus(selectedPendaftaranId, status);
        if (updated) {
            JOptionPane.showMessageDialog(this,
                "Appointment status updated
successfully.",
                "Success",
                JOptionPane.INFORMATION_MESSAGE);
            refreshTable();
        } else {
            JOptionPane.showMessageDialog(this,
                "No appointment was updated.",
                "Warning",
                JOptionPane.WARNING_MESSAGE);
        }
    } catch (RuntimeException ex) {
        JOptionPane.showMessageDialog(this,
            ex.getMessage(),
            "Database Error",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

Kode Program 3.7 Implementasi Perubahan Status Antrean pada View

Kode Program di atas merupakan implementasi proses *update* pada bagian *view*. Sistem terlebih dahulu memastikan bahwa data pendaftaran telah dipilih oleh admin, kemudian memanggil *method* `pendaftaranDAO.updateStatus()` untuk memperbarui status pendaftaran pada *database*. Apabila proses berhasil, sistem

akan memuat kembali data melalui *method* `refreshTable()` sehingga perubahan status dapat langsung ditampilkan pada halaman admin.

Object yang digunakan pada proses perubahan status tetap menggunakan *class* `Pendaftaran` sebagai model untuk menyimpan data pendaftaran selama proses berlangsung. *Class* ini memuat atribut yang sesuai dengan struktur tabel `tb_pendaftaran`, seperti identitas pasien, dokter, nomor antrian, tanggal kunjungan, keluhan, dan status pendaftaran.

```
public Pendaftaran(int pendaftaranId, int pasienId, int dokterId,
int noAntrean, Date tglKunjungan, String keluhan, String status)
{
    this.pendaftaranId = pendaftaranId;
    this.pasienId = pasienId;
    this.dokterId = dokterId;
    this.noAntrean = noAntrean;
    this.tglKunjungan = tglKunjungan;
    this.keluhan = keluhan;
    this.status = status;
}
```

Kode Program 3.8 Implementasi Model Class Pendaftaran pada Proses Update

Kode Program di atas merupakan implementasi *class* `Pendaftaran` yang digunakan sebagai model pada proses pengelolaan data pendaftaran pasien. Seluruh atribut dideklarasikan menggunakan akses *private* sehingga hanya dapat diakses melalui *method* `getter` dan `setter`. Penerapan konsep enkapsulasi membantu menjaga konsistensi data selama proses perubahan status berlangsung.

Setelah status baru dipilih oleh admin, proses berikutnya dilakukan oleh *class* `PendaftaranDAO` yang bertugas menghubungkan aplikasi dengan *database*. *Class* ini menerima id pendaftaran dan status baru dari bagian *view*, kemudian menjalankan perintah SQL untuk memperbarui data pada tabel `tb_pendaftaran`.

```
public boolean updateStatus(int pendaftaranId, String status) {
    String sql = "UPDATE tb_pendaftaran SET status = ? WHERE
pendaftaran_id = ?";

    try (Connection conn =
DatabaseConnection.getConnection();
        PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setString(1, statusForDatabase(status));
        ps.setInt(2, pendaftaranId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
```

```

        throw new RuntimeException("Failed to update
appointment status: " + e.getMessage(), e);
    }
}

```

Kode Program 3.9 Implementasi Method `updateStatus` pada `PendaftaranDAO`

Kode Program di atas merupakan implementasi *method* `updateStatus()` pada *class* `PendaftaranDAO` yang digunakan untuk memperbarui status pendaftaran pasien di *database*. *Method* ini memanfaatkan `PreparedStatement` untuk mengisi parameter pada perintah SQL `UPDATE` menggunakan ID pendaftaran dan status baru yang dipilih admin. Setelah *method* `executeUpdate()` dijalankan, sistem akan mengembalikan nilai *boolean*, yaitu *true* apabila proses pembaruan berhasil dan *false* apabila tidak ada data yang diperbarui. Nilai tersebut kemudian digunakan oleh bagian *view* untuk menentukan apakah data perlu dimuat kembali ke halaman admin.

3.1.3.2 Tampilan Interface Registration Approval dan Pengujian Update

Menu *Registration Approval* digunakan oleh admin untuk mengelola status pendaftaran pasien yang telah melakukan *book appointment*. Melalui halaman ini, admin dapat mencari data pendaftaran berdasarkan status tertentu, memilih pasien yang akan diproses, kemudian memperbarui status pendaftaran sesuai dengan kondisi yang berlaku. Pada skenario pengujian ini, admin melakukan perubahan status pendaftaran pasien Adriana Paula dengan ID pendaftaran 5 dari status *Menunggu* menjadi *Disetujui*.

Registration Approval
Track patient check-ins and manage daily appointment confirmations.

Appointment Details
Appointment ID: 5
Patient Name: Adriana Paula
Doctor: dr. Elio Sebastian
Poli: Poli Umum
Visit Date: 2026-07-06
Queue Number: 1
Complaint: Check-up rutin tekanan darah bulanan

All Appointments

All	Search	Refresh
All		
Menunggu		
Disetujui		
Sesuai		
Dibatalkan		
11	Choi Kimberly	dr. Adrian Maverick, ...
13	Valerie Clarissa	dr. Elio Sebastian
5	Adriana Paula	dr. Elio Sebastian
10	Choi Kimberly	dr. Rajwa Agustina
4	Adriana Paula	dr. Nadine Alexandra...
9	Choi Kimberly	dr. Elio Sebastian
3	Adriana Paula	dr. Adrian Maverick, ...
8	Choi Kimberly	dr. Nadine Alexandra...
2	Adriana Paula	dr. Rajwa Agustina
7	Choi Kimberly	dr. Adrian Maverick, ...
1	Adriana Paula	dr. Elio Sebastian

Gambar 3.5 Tampilan Awal Menu Registration Approval

Gambar di atas merupakan tampilan awal menu *Registration Approval* sebelum proses pembaruan status dilakukan. Pada halaman ini tersedia fitur *filter* berdasarkan status pendaftaran, yaitu *All*, *Menunggu*, dan *Disetujui*. Admin memilih *filter* *Menunggu*, kemudian menekan tombol *Search* sehingga sistem hanya menampilkan data pasien yang masih berstatus menunggu. Dari hasil pencarian tersebut, admin memilih data pendaftaran milik Adriana Paula dengan id pendaftaran 5.

Registration Approval
Track patient check-ins and manage daily appointment confirmations.

Appointment Details
Appointment ID: 5
Patient Name: Adriana Paula
Doctor: dr. Elio Sebastian
Poli: Poli Umum
Visit Date: 2026-07-06
Queue Number: 1
Complaint: Check-up rutin tekanan darah bulanan

All Appointments

ID	Patient	Doctor	Poli	Visit Date
12	Choi Kimberly	dr. Nadine Alexandra...	Poli Mata	2026-07-18
6	Adriana Paula	dr. Rajwa Agustina	Poli Gigi dan Mulut	2026-07-14
11	Choi Kimberly	dr. Adrian Maverick, ...	Poli Penyakit Dalam	2026-07-09
13	Valerie Clarissa	dr. Elio Sebastian	Poli Umum	2026-07-06
5	Adriana Paula	dr. Elio Sebastian	Poli Umum	2026-07-06
10	Choi Kimberly	dr. Rajwa Agustina	Poli Gigi dan Mulut	2026-06-30
4	Adriana Paula	dr. Nadine Alexandra...	Poli Mata	2026-06-27
9	Choi Kimberly	dr. Elio Sebastian	Poli Umum	2026-06-22
3	Adriana Paula	dr. Adrian Maverick, ...	Poli Penyakit Dalam	2026-06-18
8	Choi Kimberly	dr. Nadine Alexandra...	Poli Mata	2026-06-13
2	Adriana Paula	dr. Rajwa Agustina	Poli Gigi dan Mulut	2026-06-09
7	Choi Kimberly	dr. Adrian Maverick, ...	Poli Penyakit Dalam	2026-06-04
1	Adriana Paula	dr. Elio Sebastian	Poli Umum	2026-06-01

Gambar 3.6 Tampilan Proses Pembaruan Status Pendaftaran

Gambar di atas merupakan tampilan proses pembaruan status pendaftaran pasien melalui menu *Registration Approval*. Admin mengubah status pasien Adriana Paula dari Menunggu menjadi Disetujui menggunakan menu *dropdown*, kemudian menekan tombol *Approve* untuk menyimpan perubahan tersebut. Setelah proses *update* berhasil dilakukan, sistem memperbarui data pada database sehingga status pendaftaran pasien berubah menjadi Disetujui dan informasi terbaru dapat langsung ditampilkan pada halaman *Registration Approval*.

3.1.4 Implementasi Proses Delete

Implementasi proses *delete* merupakan proses penghapusan data dari sistem sesuai dengan hak akses pengguna. Pada sisi admin, proses *delete* terdapat pada menu *Doctor Schedule* untuk menghapus jadwal praktik dokter dan *Medical Record Entry* untuk menghapus data rekam medis berupa resep obat. Sedangkan pada sisi pasien, proses *delete* dilakukan melalui menu *My Appointment* saat membatalkan *appointment* yang telah dibuat. Pada bagian ini, implementasi proses *delete* akan dijelaskan melalui menu *Doctor Schedule*.

3.1.4.1 Potongan Kode Implementasi Doctor Schedule

Penghapusan data jadwal dokter dilakukan oleh admin melalui menu *Doctor Schedule*. Pada menu ini, admin memilih jadwal dokter yang akan dihapus, kemudian sistem menampilkan konfirmasi sebelum data dihapus dari *database*. Setelah admin menyetujui konfirmasi tersebut, sistem akan memanggil *class* *JadwalDokterDAO* untuk menjalankan proses penghapusan data.

```
private void deleteSchedule() {
    if (selectedScheduleId <= 0) {
        JOptionPane.showMessageDialog(this, "Please select a
        schedule row first.", "Validation",
        JOptionPane.WARNING_MESSAGE);
        return;
    }

    int confirm = JOptionPane.showConfirmDialog(this,
        "Delete this schedule?",
        "Confirm Delete",
        JOptionPane.YES_NO_OPTION);
    if (confirm != JOptionPane.YES_OPTION) {
        return;
    }
}
```



```

        try {
            boolean deleted =
jadwalDokterDAO.deleteSchedule(selectedScheduleId);
            if (deleted) {
                JOptionPane.showMessageDialog(this, "Schedule
deleted successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
                refreshTable();
                clearForm();
            } else {
                JOptionPane.showMessageDialog(this, "No schedule
was deleted.", "Warning", JOptionPane.WARNING_MESSAGE);
            }
        } catch (RuntimeException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(),
"Database Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

Kode Program 3.10 Implementasi Proses Delete pada Doctor Schedule

Kode Program di atas merupakan implementasi proses *delete* pada bagian View. Sistem terlebih dahulu memvalidasi apakah admin telah memilih data jadwal yang akan dihapus. Setelah admin memberikan konfirmasi, *method* `jadwalDokterDAO.deleteSchedule(selectedScheduleId)` dipanggil untuk menghapus data berdasarkan id jadwal yang dipilih. Apabila proses berhasil, sistem akan memperbarui tampilan tabel dan mengosongkan form agar data yang ditampilkan tetap sesuai dengan kondisi *database*.

Class `JadwalDokter` digunakan sebagai model yang merepresentasikan data jadwal dokter di dalam program. *Class* ini memuat atribut yang sesuai dengan struktur tabel `tb_jadwal_dokter`, seperti id jadwal, dokter, poliklinik, hari, serta jam praktik.

```

public class JadwalDokter {

    private int jadwalId;
    private int dokterId;
    private String namaDokter;
    private String namaPoli;
    private String hari;
    private String jamMulai;
    private String jamSelesai;

    public JadwalDokter() {
    }
}

```

Kode Program 3.11 Implementasi Class `JadwalDokter` sebagai Model

Kode Program di atas merupakan implementasi *class* `JadwalDokter` yang digunakan sebagai *model* untuk menyimpan data jadwal dokter selama proses berlangsung. Seluruh atribut dideklarasikan menggunakan akses *private* sehingga hanya dapat diakses melalui *method* `getter` dan `setter`. Penerapan konsep enkapsulasi membantu menjaga konsistensi data saat digunakan oleh komponen lain dalam aplikasi.

Setelah data jadwal dipilih pada bagian `View`, proses penghapusan dilakukan oleh *class* `JadwalDokterDAO` yang bertugas menghubungkan aplikasi dengan *database*. Class ini menjalankan perintah SQL untuk menghapus data jadwal dokter berdasarkan id yang diterima dari bagian `View`.

```
public boolean deleteSchedule(int jadwalId) {
    String sql = "DELETE FROM tb_jadwal_dokter WHERE
jadwal_dokter_id = ?";

    try (Connection conn =
DatabaseConnection.getConnection();
        PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setInt(1, jadwalId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        throw new RuntimeException("Failed to delete
schedule: " + e.getMessage(), e);
    }
}
```

Kode Program 3.12 Implementasi Method `deleteSchedule` pada `JadwalDokterDAO`

Kode Program di atas merupakan implementasi *method* `deleteSchedule()` pada *class* `JadwalDokterDAO` yang digunakan untuk menghapus data jadwal dokter dari *database*. *Method* ini memanfaatkan `PreparedStatement` untuk mengisi parameter pada perintah SQL `DELETE` menggunakan id jadwal yang dipilih. Klausula `WHERE jadwal_dokter_id = ?` memastikan bahwa proses penghapusan hanya dilakukan pada data yang dipilih. Setelah *method* `executeUpdate()` dijalankan, sistem akan mengembalikan nilai *true* apabila data berhasil dihapus dari *database*.

3.1.4.2 Tampilan Interface Doctor Schedule dan Pengujian Delete

Menu Doctor Schedule digunakan oleh admin untuk mengelola data jadwal praktik dokter, mulai dari menambahkan, mengubah, hingga menghapus

jadwal yang sudah tidak diperlukan. Pada skenario pengujian delete ini, admin melakukan penghapusan jadwal praktik milik dr. Adrian Maverick melalui data yang tersedia pada tabel jadwal dokter. Proses penghapusan diawali dengan memilih data jadwal yang akan dihapus, kemudian sistem menampilkan informasi jadwal pada form sehingga admin dapat memastikan data yang dipilih sudah sesuai sebelum melakukan penghapusan.

ID	Doctor ID	Doctor	Poli	Day	Start Time	End Time
3	3	dr. Adrian Maverick, ...	Poli Penyakit Dalam	Kamis	13:00:00	17:00:00
1	1	dr. Elio Sebastian	Poli Umum	Senin	08:00:00	12:00:00
4	4	dr. Nadine Alexandra...	Poli Mata	Sabtu	08:00:00	12:00:00
2	2	drg. Rajwa Agustina	Poli Gigi dan Mulut	Selasa	09:00:00	13:00:00

Gambar 3.7 Tampilan Menu Doctor Schedule

Gambar di atas merupakan tampilan menu Doctor Schedule saat admin melakukan penghapusan data jadwal dokter. Admin terlebih dahulu memilih baris jadwal milik dr. Adrian Maverick pada tabel yang berada di panel sebelah kanan. Setelah baris dipilih, informasi jadwal akan ditampilkan secara otomatis pada form di panel sebelah kiri. Selanjutnya, admin menekan tombol Delete, kemudian sistem menghapus data jadwal dokter tersebut dari database dan memperbarui tampilan tabel sehingga data yang telah dihapus tidak lagi ditampilkan.

DAFTAR PUSTAKA

- Duha, W. S., Hakim, A. N., Suci, C. N., & Niqotaini, Z. (2025). Analisis dan Perancangan Sistem Informasi Layanan Rumah Sakit Berbasis Web Menggunakan UML. *Jurnal PROSISKO*, 10-22.
- Fadil, I., & Ruhiat, A. (2018). Sistem Informasi Pendaftaran dan Antrian Pasien Pada Klinik Dokter Menggunakan Komunikasi Data Internet. *Jurnal Ilmu-ilmu Informatika dan Manajemen STMIK* , 83-92.
- Satriawan, I. B., Sanjaya, I. S., Maurita, E., Christiano, M., Fauzan, D. A., & Akbar, F. A. (2024). Pengembangan Program Sistem Manajemen Klinik Berbasis Desktop. *Seminar Nasional Informatika Bela Negara (SANTIKA)* , 2747-0563 .
- Wardani, C. A., Apriyanto, & Susilowati, S. (2024). Perancangan Sistem Informasi Pendaftaran Pasien pada Klinik Adisty Bogor. *Riset dan E-Jurnal Manajemen Informatika Komputer*, 1029-1040.
doi:10.33395/remik.v8i4.14107